

Los desafíos de las clases 6 a 11, resueltos como un proyecto real

Cada desafío de la plataforma desarrollado en un editor de código: carpeta de proyecto, **index.html** y **css/estilos.css** por separado, con la explicación completa de cada decisión.

Clase 6	Panel de estado con CSS — <i>Reglas, selectores y :hover</i>
Clase 7	Planes de servicio — <i>Box Model y escala de espaciado</i>
Clase 8	Tema automático y selector manual — <i>Variables CSS y modo oscuro</i>
Clase 9	Organizar una aplicación de música — <i>Flexbox</i>
Clase 10	Organizar un panel de proyectos — <i>CSS Grid</i>
Clase 11	Adaptar un panel de seguimiento — <i>Diseño responsivo</i>

Contenido

CAPÍTULO 0	Del editor de la plataforma al proyecto real	3
	Estructura de carpetas, <code><link></code> , rutas, VS Code y errores típicos	
CAPÍTULO 1	Clase 6 — Panel de estado con CSS	9
	Reglas, selectores de clase y descendientes, <code>:hover</code>	
CAPÍTULO 2	Clase 7 — Planes de servicio	19
	Box Model, <code>border-box</code> , padding vs. margin, escala de espaciado	
CAPÍTULO 3	Clase 8 — Tema automático y selector manual	28
	Variables CSS, modo oscuro, contraste y un archivo JavaScript	
CAPÍTULO 4	Clase 9 — Organizar una aplicación de música	39
	Flexbox: contenedores, ejes, <code>flex</code> , <code>wrap</code> y <code>gap</code>	
CAPÍTULO 5	Clase 10 — Organizar un panel de proyectos	52
	CSS Grid: áreas, <code>fr</code> , <code>auto-fit</code> y <code>span</code>	
CAPÍTULO 6	Clase 11 — Adaptar un panel de seguimiento	62
	Media queries, mobile first, <code>viewport</code> y breakpoints	

Cada capítulo es independiente: se puede leer suelto. El Capítulo 0 explica el mecanismo de archivos separados que después se usa en todos los demás, así que conviene leerlo primero.

CAPÍTULO 0 · ANTES DE EMPEZAR

Del editor de la plataforma al proyecto real

Cómo se traduce lo que venís haciendo en las tres pestañas de la plataforma a una carpeta de proyecto con archivos separados, igual a la que usa cualquier desarrollador en su computadora.

Por qué este material

En la plataforma UDI el desafío de cada clase se resuelve en un editor con tres pestañas: **HTML**, **CSS** y **JS**. Esas tres pestañas son cómodas para aprender —no hay que preocuparse por archivos ni por rutas— pero esconden un paso que en el trabajo real hay que hacer a mano: **los archivos son tres archivos distintos, guardados en una carpeta, y es el HTML el que tiene que ir a buscar a los otros dos**.

Este documento toma los seis desafíos de las clases 6 a 11 —los mismos, sin cambiar una coma— y los muestra desarrollados como se harían en un editor de código: carpeta de proyecto, `index.html` por un lado, `css/estilos.css` por el otro, y todo explicado paso a paso.

Cómo usar este material

La plataforma no cambia: los desafíos se siguen entregando ahí. Esto es material complementario para leer al lado del editor, o para armar el proyecto en la computadora y ver el mismo resultado fuera de la plataforma.

La diferencia en una imagen

Del lado izquierdo, lo que ves hoy. Del lado derecho, lo mismo en un proyecto real.

En la plataforma	En un proyecto real
Pestaña HTML	Archivo <code>index.html</code>
Pestaña CSS	Archivo <code>css/estilos.css</code>
Pestaña JS	Archivo <code>js/app.js</code>
La plataforma une las tres pestañas sola y te muestra el resultado	Vos escribís la línea que conecta el CSS y el JS con el HTML
Se ve en el panel de vista previa	Se ve abriendo <code>index.html</code> en el navegador
El trabajo queda guardado en tu cuenta	El trabajo queda en una carpeta de tu computadora

Todo lo demás —los selectores, las propiedades, el Box Model, Flexbox, Grid, las media queries— es **exactamente igual**. Lo único que se agrega es la conexión entre archivos.

La estructura de carpetas que vamos a usar

Para todos los desafíos de este documento usamos siempre la misma estructura. Es la convención más común en proyectos chicos y medianos:

```
desafio-clase-06/      ← la carpeta del proyecto
├─ index.html          ← la página (estructura y contenido)
├─ css/
│   └─ estilos.css     ← todos los estilos
```

Cuando además hay JavaScript, se agrega una tercera carpeta:

```
desafio-clase-08/
├─ index.html
├─ css/
│   └─ estilos.css
├─ js/
│   └─ tema.js
```

Tres reglas de oro para los nombres

1. **Todo en minúscula.** `Estilos.css` y `estilos.css` son dos archivos distintos para un servidor web. En Windows suele funcionar igual; cuando el sitio se sube a un servidor real, deja de funcionar. Empezá bien desde el principio.
2. **Sin espacios ni acentos.** `mi pagina.html` se convierte en `mi%20pagina.html` en la URL. Usá guiones: `mi-pagina.html`.
3. **La página principal siempre se llama `index.html`.** Es el nombre que los servidores buscan por defecto cuando alguien entra a una dirección sin especificar el archivo.

La línea que conecta todo: `<link>`

Esta es la línea más importante de todo el documento. Va dentro del `<head>` y es la que le dice al navegador dónde está la hoja de estilos:

```
<link rel="stylesheet" href="css/estilos.css">
```

Vale la pena leerla parte por parte, porque cada pedazo hace algo:

Parte	Qué significa
<code><link></code>	Una etiqueta que vincula un recurso externo a esta página. No muestra nada por sí sola. No se cierra: no existe <code></link></code> .

Parte	Qué significa
<code>rel="stylesheet"</code>	<i>Relación</i> : qué es ese archivo que estoy vinculando. <code>stylesheet</code> = hoja de estilos. Si esto falta o está mal escrito, el navegador descarga el archivo pero no lo aplica .
<code>href="css/estilos.css"</code>	<i>Hypertext reference</i> : la ruta hasta el archivo, contada desde donde está el HTML .

Dónde va exactamente

Dentro del `<head>`, después del `<title>`. Si lo ponés en el `<body>` suele funcionar igual, pero el navegador dibuja la página primero sin estilos y después los aplica: se ve un parpadeo feo. En el `<head>` los estilos llegan antes de que se dibuje nada.

Cómo se leen las rutas

La ruta del `href` se lee **parada en el archivo que la escribe**. Si el `index.html` está en la raíz del proyecto y el CSS está en una subcarpeta `css/`, la ruta es `css/estilos.css`: "entrá a la carpeta `css` y buscá ahí el archivo `estilos.css`".

Se escribe	Se lee
<code>estilos.css</code>	En la misma carpeta que el HTML.
<code>css/estilos.css</code>	Entrá a la carpeta <code>css</code> (que está al lado del HTML) y buscá ahí.
<code>../estilos.css</code>	Subí un nivel (a la carpeta que contiene a esta) y buscá ahí.
<code>../../img/logo.png</code>	Subí dos niveles, entrá a <code>img</code> y buscá <code>logo.png</code> .
<code>/css/estilos.css</code>	Desde la raíz del sitio . Ojo: abriendo el archivo con doble clic esto casi nunca funciona. En proyectos chicos, evitala.

La línea del JavaScript: `<script>`

Cuando hay JavaScript, la conexión es parecida pero con dos diferencias importantes:

```
<script src="js/app.js"></script>
```

- Se usa `src` (source), no `href`.
- Sí lleva etiqueta de cierre**, aunque esté vacía. `<script src="...">` sin `</script>` rompe la página entera.
- Va al final del `<body>`, justo antes de `</body>`. Así el JavaScript se ejecuta cuando el HTML ya existe: si va en el `<head>`, el script busca elementos que todavía no fueron creados y falla.

La plantilla completa

Este es el esqueleto del que arranca cualquier página. Vale la pena aprendérselo de memoria: es el mismo siempre.

```
index.html — plantilla base HTML
1  <!DOCTYPE html>
2  <html lang="es">
3    <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Título de la página</title>
7      <link rel="stylesheet" href="css/estilos.css">
8    </head>
9    <body>
10
11      <!-- Aquí va todo el contenido visible -->
12
13      <script src="js/app.js"></script>
14    </body>
15  </html>
```

Dos etiquetas del `<head>` que a veces se pasan por alto:

- `<meta charset="UTF-8">` — le dice al navegador que el archivo usa la codificación UTF-8. Sin esto los acentos y las eñes se ven como `Ã` y `±`.
- `<meta name="viewport" ...>` — le dice al celular que use su ancho real y no que finja ser una pantalla de escritorio y achique todo. Sin esta línea, las media queries de la clase 11 no funcionan en un teléfono.

Cómo armar el proyecto, paso a paso

Con Visual Studio Code (recomendado)

1. Creá una carpeta en tu computadora con el nombre del desafío, por ejemplo `desafio-clase-06`.
2. Abrió VS Code y andá a **Archivo** → **Abrir carpeta...** y elegí esa carpeta. En el panel de la izquierda vas a ver el nombre de la carpeta: ese panel es tu proyecto.
3. Con el botón "nuevo archivo" del panel izquierdo, creá `index.html`. **Escribí la extensión:** sin el `.html`, VS Code no colorea nada y el navegador no lo entiende.
4. Con el botón "nueva carpeta", creá `css`. Hacé clic sobre esa carpeta y creá adentro `estilos.css`.
5. Pegá el HTML de arranque en `index.html` y el CSS de arranque en `css/estilos.css`.
6. Verificá que en el `<head>` del HTML esté la línea del `<link>`.
7. Guardá los dos archivos: `Ctrl+S` (o `Cmd+S` en Mac).

Cómo ver el resultado

Opción simple: hacé doble clic en `index.html` desde el explorador de archivos. Se abre en el navegador. Cada vez que guardes un cambio, volvé al navegador y apretá `F5` para recargar.

Opción cómoda: instalá la extensión **Live Server** en VS Code (ícono de extensiones, buscar "Live Server", Instalar). Después, clic derecho sobre `index.html` → **Open with Live Server**. El navegador se recarga solo cada vez que guardás. Es lo más parecido al panel de vista previa de la plataforma.

Las herramientas de desarrollo son parte del oficio

Con la página abierta, `F12` (o clic derecho → **Inspeccionar**) abre las DevTools del navegador. En la pestaña **Elements** podés hacer clic en cualquier elemento y ver, a la derecha, **qué reglas CSS se le están aplicando y cuáles fueron tachadas por otra regla más específica**. Es la forma más rápida de entender por qué algo no se ve como esperabas. En la pestaña **Console** aparecen los errores de JavaScript y los archivos que no se pudieron cargar.

Los cinco errores que vas a cometer la primera vez

Todos los cometimos. Van con su síntoma, para reconocerlos rápido.

Síntoma	Causa probable	Solución
La página se ve en Times New Roman, sin colores, todo apilado	El CSS no está llegando	Revisá el <code><link></code> : que exista, que diga <code>rel="stylesheet"</code> y que la ruta coincida letra por letra con el nombre real del archivo.
El CSS no llega y la ruta parece bien	La carpeta se llama <code>CSS</code> y la ruta dice <code>css</code> , o el archivo es <code>estilo.css</code> y la ruta dice <code>estilos.css</code>	Comparalos con el panel de VS Code al lado. Un solo carácter de diferencia alcanza.
Guardaste pero no cambia nada	No guardaste (el nombre de la pestaña en VS Code tiene un punto en vez de la X), o no recargaste	<code>Ctrl+S</code> y después <code>F5</code> . Si igual no cambia, <code>Ctrl+Shift+R</code> para forzar la recarga sin caché.
El archivo aparece como texto plano en el navegador	Se guardó sin extensión o como <code>.txt</code>	Renombralo asegurándote de que termine en <code>.html</code> .
El JavaScript no hace nada	Falta <code></script></code> , o el <code><script></code> está en el <code><head></code>	Cerrá la etiqueta y movela al final del <code><body></code> . Mirá la pestaña Console.

Cómo está organizado cada capítulo

Los seis capítulos que siguen tienen todos la misma estructura, así que una vez que agarrás el ritmo del primero, los demás se leen solos:

Sección	Qué vas a encontrar
Ficha del desafío	De qué clase viene, qué se practica y qué archivos tiene el proyecto.
Estructura del proyecto	El árbol de carpetas exacto para armar en VS Code.
index.html	El archivo completo, idéntico al de la plataforma más la línea del <code><link></code> .
La consigna	Copiada textual de la plataforma, para no perder de vista qué se pide.
Punto de partida del CSS	El archivo tal como arranca, antes de resolver nada.
Cómo lo encara un desarrollador	La estrategia antes de escribir la primera línea.
Paso a paso	Cada requisito resuelto y explicado: qué se escribe y por qué.
Solución completa	El <code>css/estilos.css</code> final, comentado, listo para comparar.
Cómo probarlo	Qué mirar en el navegador para saber que está bien.
Errores frecuentes	Los tropiezos típicos de ese tema en particular.

Una aclaración importante sobre la solución

La solución completa que aparece al final de cada capítulo es **una** solución, no **la** solución. En CSS casi siempre hay más de un camino correcto. Si tu versión se ve igual y podés explicar por qué escribiste cada regla, tu versión está bien. Usá la solución de referencia para comparar criterios, no para copiar.

CAPÍTULO 1 · CLASE 6

Desafío: Panel de estado con CSS

Un tablero que muestra si tres servicios están funcionando. Todo el desafío se resuelve escribiendo CSS: el HTML no se toca. Es el primer proyecto donde el CSS vive en su propio archivo.

Ficha del desafío

Clase	Clase 6 — Introducción a CSS: reglas y selectores
Desafío	Panel de estado con CSS
Qué se practica	Anatomía de una regla CSS, selectores de etiqueta, de clase y descendientes, la pseudoclase <code>:hover</code> , y el criterio de no repetir código
Archivos del proyecto	<code>index.html</code> + <code>css/estilos.css</code>
Qué se toca	Solo el CSS. El HTML viene dado y no se modifica.

Estructura del proyecto

```
desafio-clase-06/  
├─ index.html  
└─ css/  
    └─ estilos.css
```

Dos archivos, una carpeta. Es la estructura más chica posible de un sitio real y es la que vamos a repetir en casi todos los desafíos.

El archivo `index.html`

Es el HTML que ya conocés de la plataforma, con una sola línea agregada: el `<link>` que va a buscar la hoja de estilos. También agregamos la etiqueta `viewport`, que la plataforma pone sola pero en un proyecto real hay que escribir.

index.html

HTML

```
1 <!DOCTYPE html>  
2 <html lang="es">
```

```
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Estado</title>
7
8   <!-- ESTA es la línea nueva respecto de la plataforma:
9        conecta este HTML con la hoja de estilos. -->
10  <link rel="stylesheet" href="css/estilos.css">
11 </head>
12 <body>
13   <header>
14     <h1>Estado de servicios</h1>
15     <nav>
16       <a href="#servicios">Servicios</a>
17       <a href="#contacto">Contacto</a>
18     </nav>
19   </header>
20   <main id="servicios">
21     <article class="tarjeta correcto">
22       <h2>API</h2>
23       <p>Operativa</p>
24     </article>
25     <article class="tarjeta advertencia">
26       <h2>Correo</h2>
27       <p>Con demoras</p>
28     </article>
29     <article class="tarjeta error">
30       <h2>Pagos</h2>
31       <p>Interrumpido</p>
32     </article>
33   </main>
34   <footer id="contacto">
35     <p>soporte@equipo.test</p>
36   </footer>
37 </body>
38 </html>
```

Leé el HTML antes de escribir CSS

Antes de tocar una sola propiedad, mirá qué te da el HTML. Acá lo importante es que cada `<article>` tiene **dos clases separadas por un espacio**: `class="tarjeta correcto"`. Eso no es un error ni una clase con nombre raro: son dos clases distintas aplicadas al mismo elemento. Una dice "esto es una tarjeta" y la otra dice "y además está en estado correcto". Ese detalle es toda la estrategia de la solución.

La consigna

CONSIGNA TAL COMO APARECE EN LA PLATAFORMA

OBJETIVO

Diseñar desde cero el CSS de un panel usando los selectores y propiedades de la clase.

DÓNDE TRABAJAR

Pestaña CSS. No cambies HTML.

REQUISITOS

1. body con tipografía, fondo, color e interlineado legibles.
2. header con fondo oscuro y texto claro.
3. nav a sin subrayado y con :hover.
4. .tarjeta con fondo, borde, radio y padding.
5. .correcto, .advertencia y .error con colores diferentes.
6. Selectores descendientes para h2 y p dentro de .tarjeta.
7. Footer visualmente secundario.

CÓMO SABER QUE TERMINASTE

La jerarquía se entiende, las tres tarjetas comparten estructura, cada estado se diferencia, los enlaces reaccionan y no hay estilos inline.

DESAFÍO OPCIONAL

No repitas las mismas declaraciones en cada tarjeta.

Punto de partida del CSS

El archivo arranca prácticamente vacío. Todo lo que sigue lo escribís vos.

css/estilos.css — punto de partida

CSS

```
1  /* =====
2     Desafío Clase 6 — Panel de estado
3     Todo el CSS del proyecto vive en este archivo.
4     ===== */
5
6  /* Escribí todo el CSS acá */
```

Cómo lo encara un desarrollador

Antes de escribir la primera llave, conviene tener dos decisiones tomadas.

Primera: el orden de arriba hacia abajo. El CSS se escribe de lo más general a lo más específico. Primero `body` (que afecta a todo), después las zonas grandes (`header`, `main`, `footer`), después los componentes (`.tarjeta`) y por último las variantes (`.correcto`, `.error`). Ese orden no es capricho: en CSS, **a igual especificidad, gana la regla que está más abajo**. Escribir de general a específico hace que las excepciones caigan naturalmente después de las reglas base.

Segunda: separar lo que se comparte de lo que cambia. Las tres tarjetas tienen el mismo fondo, el mismo borde, el mismo radio y el mismo padding. Lo único distinto entre ellas es el color. Si escribimos tres reglas completas, cualquier cambio futuro (por ejemplo, más padding) hay que hacerlo tres veces y es cuestión de tiempo que una quede distinta. Por eso: una regla `.tarjeta` con todo lo compartido, y tres reglas mínimas de color. Eso es exactamente lo que pide el desafío opcional.

Paso a paso

Paso 1 — La base de lectura (requisito 1)

La regla de `body` es la que fija el "aire" de toda la página. Todo lo que se escriba acá lo heredan los elementos de adentro, salvo que alguno lo pise.

```
1  body {  
2    font-family: Arial, sans-serif;  
3    background: #f5f5f7;  
4    color: #1d1d1f;  
5    line-height: 1.6;  
6    margin: 0;  
7  }
```

- `font-family: Arial, sans-serif;` — la lista se lee de izquierda a derecha: usá Arial; si no está instalada, usá cualquier tipografía sin serifs. Siempre se cierra con una familia genérica.
- `line-height: 1.6;` — el alto de cada renglón. Va **sin unidad**: es un multiplicador del tamaño de letra. Con `1` los renglones se tocan; `1.6` los separa y el texto se vuelve cómodo de leer. Es probablemente la propiedad que más mejora una página con menos esfuerzo.
- `margin: 0;` — el navegador le pone al `body` un margen de unos 8px por defecto. Si no lo sacamos, la cabecera oscura queda con una franja blanca alrededor y no llega a los bordes de la ventana.

Paso 2 — La cabecera (requisito 2)

La cabecera tiene que ser lo primero que el ojo encuentra. El recurso más directo es invertir los colores: fondo oscuro, texto claro.

```
1  header {  
2    background: #1d1d1f;  
3    color: white;  
4    padding: 24px 32px;  
5  }  
6  
7  header h1 {  
8    margin: 0 0 8px 0;  
9    font-size: 26px;  
10 }
```

`padding: 24px 32px;` con dos valores significa **24px arriba y abajo, 32px a los lados**. Es la forma abreviada; la vas a ver todo el tiempo. Y `header h1` es el primer **selector descendiente** del archivo: se lee "los `h1` que estén adentro de un `header`". Le sacamos el margen de arriba que trae por defecto para que el título arranque justo donde termina el padding.

Paso 3 — Los enlaces del menú (requisito 3)

```
1  nav a {
2    color: #9ecbff;
3    text-decoration: none;
4    margin-right: 16px;
5  }
6
7  nav a:hover {
8    color: white;
9    text-decoration: underline;
10 }
```

Dos ideas conviven acá. `nav a` apunta solo a los enlaces del menú: si mañana hay un enlace en el pie, este no lo toca. Y `nav a:hover` usa una **pseudoclase**: los dos puntos no separan propiedad y valor, marcan un *estado* del elemento. Se lee "los enlaces del nav, cuando el mouse está encima".

Por qué el hover no es opcional

Al sacar el subrayado, el enlace deja de parecer un enlace. El `:hover` devuelve esa señal: el elemento reacciona, entonces se puede tocar. Cada vez que le saques a un elemento su apariencia por defecto, pregúntate qué señal estás sacando con ella y con qué la reemplazás.

Paso 4 — La tarjeta compartida (requisito 4)

Acá está el corazón del desafío: **una sola regla** con todo lo que las tres tarjetas tienen en común.

```
1  main {
2    padding: 24px 32px;
3  }
4
5  .tarjeta {
6    background: white;
7    border: 1px solid #d2d2d7;
8    border-left: 5px solid #d2d2d7;
9    border-radius: 10px;
10   padding: 16px 20px;
11   margin-bottom: 12px;
12 }
```

El truco fino es el doble borde. `border: 1px solid #d2d2d7;` pone un borde de 1px en los cuatro lados. La línea siguiente, `border-left: 5px solid #d2d2d7;`, pisa **solo el izquierdo** y lo hace más grueso. Ese borde grueso todavía es gris: es un espacio reservado que cada estado va a pintar con su color en el paso 6.

Paso 5 — El contenido de las tarjetas (requisito 6)

```
1 .tarjeta h2 {
2   margin: 0 0 4px 0;
3   font-size: 17px;
4 }
5
6 .tarjeta p {
7   margin: 0;
8   font-size: 14px;
9   font-weight: bold;
10 }
```

Otra vez selectores descendientes, y otra vez escritos una sola vez para las tres tarjetas. `.tarjeta h2` alcanza a los tres títulos, y `.tarjeta p` a los tres estados. Los `margin: 0` son necesarios porque `h2` y `p` traen márgenes propios del navegador que, dentro de una caja con padding, se suman y descolocan todo.

Paso 6 — Los tres estados (requisito 5)

Recién ahora aparecen las diferencias, y son mínimas: dos declaraciones por estado.

```
1 .correcto {
2   border-left-color: #18794e;
3 }
4 .correcto p {
5   color: #18794e;
6 }
7
8 .advertencia {
9   border-left-color: #b54708;
10 }
11 .advertencia p {
12   color: #b54708;
13 }
14
15 .error {
16   border-left-color: #b42318;
17 }
18 .error p {
19   color: #b42318;
20 }
```

Fijate en `border-left-color`: no reescribimos el borde entero, solo su color. El grosor y el estilo ya los puso `.tarjeta`. Y `.correcto p` pinta el texto del estado, no toda la tarjeta.

Por qué esto funciona y no hay conflicto

El elemento tiene `class="tarjeta correcto"`, así que le aplican **las dos reglas a la vez**. No compiten: `.tarjeta` define seis propiedades y `.correcto` define una sola. La única que aparece en las dos sería el color del borde izquierdo, y ahí gana `.correcto` porque está escrita más abajo. Por eso el orden importa: si pusieras las tres reglas de estado *antes* de `.tarjeta`, el gris las pisaría y las tres tarjetas se verían iguales.

Los colores no son decorativos: verde, ámbar y rojo son el código universal de estado. Además, cada tarjeta dice en palabras qué le pasa ("Operativa", "Con demoras", "Interrumpido"), así que la información no depende **solo** del color — eso importa para quien no distingue colores.

Paso 7 — El pie (requisito 7)

```
1 footer {
2   border-top: 1px solid #d2d2d7;
3   padding: 16px 32px;
4   color: #6e6e73;
5   font-size: 13px;
6 }
```

"Secundario" en diseño se logra con tres recursos combinados: menos tamaño, menos contraste (gris en vez de negro) y una línea que lo separa del contenido principal. No hace falta nada más.

Solución completa

El archivo `css/estilos.css` terminado. Los comentarios están puestos a propósito: un CSS real se lee muchas más veces de las que se escribe.

css/estilos.css — solución completa

CSS

```
1  /* =====
2   Desafío Clase 6 – Panel de estado
3   css/estilos.css
4   -----
5   ESTRATEGIA
6   El HTML tiene tres <article class="tarjeta correcto|advertencia|error">.
7   Cada tarjeta lleva DOS clases: .tarjeta define todo lo que comparten
8   (fondo, borde, radio, padding) y la segunda clase solo cambia el color.
9   Así no se repiten las mismas declaraciones tres veces, que es el
10  desafío opcional de la consigna.
11  ===== */
12
13  /* ---- REQUISITO 1: base de lectura de toda la página ---- */
14  body {
15    font-family: Arial, sans-serif;
16    background: #f5f5f7;
17    color: #1d1d1f;
18    line-height: 1.6;
19    margin: 0;
20  }
21
22  /* ---- REQUISITO 2: cabecera ---- */
23  /* Fondo oscuro con texto claro. El contraste alto la separa del
24     resto y la vuelve el ancla visual de la página. */
25  header {
26    background: #1d1d1f;
27    color: white;
28    padding: 24px 32px;
29  }
30
```

```

31 header h1 {
32     margin: 0 0 8px 0;
33     font-size: 26px;
34 }
35
36 /* ---- REQUISITO 3: enlaces del menú ----- */
37 /* Selector descendiente: solo los <a> que están dentro de un <nav>. */
38 nav a {
39     color: #9ecbff;
40     text-decoration: none;
41     margin-right: 16px;
42 }
43
44 nav a:hover {
45     color: white;
46     text-decoration: underline;
47 }
48
49 /* ---- REQUISITO 4: la estructura común de las tarjetas ----- */
50 main {
51     padding: 24px 32px;
52 }
53
54 /* Escrita UNA sola vez. El border-left más grueso deja el lugar
55    donde después cada estado pone su color. */
56 .tarjeta {
57     background: white;
58     border: 1px solid #d2d2d7;
59     border-left: 5px solid #d2d2d7;
60     border-radius: 10px;
61     padding: 16px 20px;
62     margin-bottom: 12px;
63 }
64
65 /* ---- REQUISITO 6: contenido interno de las tarjetas ----- */
66 /* Selectores descendientes: sirven para las tres por igual. */
67 .tarjeta h2 {
68     margin: 0 0 4px 0;
69     font-size: 17px;
70 }
71
72 .tarjeta p {
73     margin: 0;
74     font-size: 14px;
75     font-weight: bold;
76 }
77
78 /* ---- REQUISITO 5: lo único que cambia entre estados ----- */
79 /* Verde = todo bien, ámbar = atención, rojo = caído.
80    Cada regla pinta el texto y la barra lateral izquierda. */
81 .correcto {
82     border-left-color: #18794e;
83 }
84 .correcto p {
85     color: #18794e;
86 }
87
88 .advertencia {
89     border-left-color: #b54708;
90 }
91 .advertencia p {

```

```

92     color: #b54708;
93 }
94
95 .error {
96     border-left-color: #b42318;
97 }
98 .error p {
99     color: #b42318;
100 }
101
102 /* ---- REQUISITO 7: pie de página ----- */
103 /* Información secundaria: más chica, gris y separada por una línea. */
104 footer {
105     border-top: 1px solid #d2d2d7;
106     padding: 16px 32px;
107     color: #6e6e73;
108     font-size: 13px;
109 }

```

Cómo probarlo

1. Abrió `index.html` en el navegador (doble clic, o Live Server).
2. La cabecera oscura tiene que llegar hasta los bordes de la ventana, sin franja blanca alrededor.
3. Pasá el mouse por "Servicios" y "Contacto": el color tiene que cambiar y aparecer el subrayado.
4. Las tres tarjetas tienen que verse **del mismo tamaño y con la misma forma**, cambiando solo la barra de color de la izquierda y el color del texto de estado.
5. Apretá `F12`, hacé clic sobre una tarjeta en la pestaña Elements y mirá el panel de estilos: vas a ver las reglas `.tarjeta` y `.correcto` aplicándose juntas, y el `border-left-color` de `.tarjeta` tachado. Esa tachadura es la cascada funcionando.

Errores frecuentes

Qué pasa	Por qué	Cómo se arregla
Las tres tarjetas se ven grises	Las reglas <code>.correcto</code> , <code>.advertencia</code> y <code>.error</code> están escritas antes que <code>.tarjeta</code>	Movelas debajo. A igual especificidad gana la última.
La cabecera tiene una franja blanca alrededor	Falta <code>margin: 0</code> en el <code>body</code>	Agregalo.
El texto de las tarjetas queda pegado al borde	Falta el <code>padding</code> en <code>.tarjeta</code>	El padding es el espacio adentro de la caja; el margin es el de afuera.

Qué pasa	Por qué	Cómo se arregla
El <code>:hover</code> no hace nada	Se escribió <code>nav a :hover</code> con espacio, o <code>nav: hover a</code>	<code>nav a: hover</code> , todo pegado. Un espacio de más cambia por completo el significado.
Los títulos de las tarjetas quedan muy separados	No se anuló el margen por defecto de <code>h2</code>	<code>.tarjeta h2 { margin: 0 0 4px 0; }</code>

Para pensar antes de entregar

¿Podrías agregar un cuarto estado —por ejemplo `.mantenimiento` en azul— escribiendo solo dos declaraciones de CSS? Si la respuesta es sí, resolviste bien el desafío opcional. Si tendrías que copiar la regla entera de la tarjeta, todavía hay repetición para sacar.

CAPÍTULO 2 · CLASE 7

Desafío: Planes de servicio

Tres tarjetas de precios del mismo tamaño, una de ellas destacada, con todos los espacios salidos de una escala de 8 en 8. El desafío del Box Model: que nada sobresalga y que todo espacio se pueda justificar.

Ficha del desafío

Clase	Clase 7 — Box Model: espacio, bordes y dimensiones
Desafío	Planes de servicio
Qué se practica	<code>box-sizing: border-box</code> , padding vs. margin, <code>width</code> y <code>max-width</code> , centrado con <code>margin: auto</code> y el concepto de escala de espaciado
Archivos del proyecto	<code>index.html</code> + <code>css/estilos.css</code>
Qué se toca	Solo el CSS.

Estructura del proyecto

```
desafio-clase-07/
├─ index.html
├─ css/
│   └─ estilos.css
```

El archivo `index.html`

index.html	HTML
<pre> 1 <!DOCTYPE html> 2 <html lang="es"> 3 <head> 4 <meta charset="UTF-8"> 5 <meta name="viewport" content="width=device-width, initial-scale=1.0"> 6 <title>Planes</title> 7 <link rel="stylesheet" href="css/estilos.css"> 8 </head> 9 <body> 10 <h1>Planes de servicio</h1></pre>	

```
11
12     <article class="plan">
13       <h2>Inicial</h2>
14       <ul>
15         <li>1 proyecto</li>
16         <li>Soporte básico</li>
17       </ul>
18       <button>Elegir</button>
19     </article>
20
21     <article class="plan recomendado">
22       <h2>Equipo</h2>
23       <ul>
24         <li>10 proyectos</li>
25         <li>Soporte prioritario</li>
26       </ul>
27       <button>Elegir</button>
28     </article>
29
30     <article class="plan">
31       <h2>Empresa</h2>
32       <ul>
33         <li>Proyectos ilimitados</li>
34         <li>Soporte dedicado</li>
35       </ul>
36       <button>Elegir</button>
37     </article>
38   </body>
39 </html>
```

Otra vez el patrón de dos clases: `class="plan recomendado"`. El segundo `<article>` es "un plan, y además el recomendado". El HTML no dice nada de cómo se ve: dice **qué es** cada cosa. Eso está bien y es lo que buscamos.

La consigna

CONSIGNA TAL COMO APARECE EN LA PLATAFORMA

OBJETIVO

Diseñar desde cero el CSS de tres planes usando un sistema de cajas y espacios.

DÓNDE TRABAJAR

Pestaña CSS. No cambies HTML.

REQUISITOS

1. Aplicá border-box global.
2. body con max-width y centrado.
3. .plan con ancho adaptable, max-width, padding, borde, radio y separación.
4. Los tres planes deben compartir la misma regla base.
5. .recomendado debe diferenciarse con color de borde y fondo.
6. Listas y botón deben tener espacios coherentes usando 8, 16, 24 o 32px.
7. El botón no debe superar el ancho interno.

CÓMO SABER QUE TERMINASTE

Las tres cajas tienen estructura consistente, ninguna sobresale, el plan recomendado se identifica sin romper el diseño y podés justificar cada padding y margin.

DESAFÍO OPCIONAL

No uses valores de espaciado fuera de 8, 16, 24 y 32px.

Punto de partida del CSS

css/estilos.css – punto de partida

CSS

```
1  /* =====
2     Desafío Clase 7 – Planes de servicio
3     ===== */
4
5  /* Escribí todo el CSS acá */
```

Cómo lo encara un desarrollador

Este desafío parece de diseño y en realidad es de **medidas**. Hay dos ideas que hay que tener claras antes de escribir.

Idea 1: qué mide `width` en realidad

Por defecto, el navegador entiende `width: 300px` como "300px de contenido", y después le **suma** el padding y el borde por afuera. Una caja de 300px con 24px de padding a cada lado y 1px de borde mide en pantalla 350px. Eso se llama `content-box` y es la fuente número uno de diseños que se desbordan.

Con `box-sizing: border-box` la cuenta cambia: `width: 300px` pasa a significar "300px en total, padding y borde incluidos", y el contenido se acomoda adentro. Es muchísimo más intuitivo y es lo que hace todo el mundo. Por eso la primera regla del archivo lo aplica **a todos los elementos** con el selector universal `*`.

Padding o margin: la pregunta de siempre

- **Padding** es espacio *adentro* de la caja: empuja el contenido hacia adentro y el fondo de la caja llega hasta el borde. Si querés que el texto no toque el borde de la tarjeta, es padding.
- **Margin** es espacio *afuera*: separa esta caja de las vecinas y no tiene fondo. Si querés separar una tarjeta de la siguiente, es margin.

Idea 2: la escala de espaciado

El desafío opcional pide usar solamente 8, 16, 24 y 32px. No es un capricho: es una técnica real. Cuando cada espacio se elige "a ojo" aparecen 13px acá, 17px allá y 22px más abajo, y el resultado se ve desprolijo aunque nadie sepa explicar por qué. Con una escala de múltiplos de 8 todo entra en un ritmo visual y **cada decisión se puede justificar**: 8 para lo pegado, 16 para lo relacionado, 24 para separar bloques, 32 para separar secciones.

Paso a paso

Paso 1 — border-box global (requisito 1)

```
1  * {  
2    box-sizing: border-box;  
3  }
```

El `*` es el **selector universal**: alcanza a todos los elementos de la página. Esta regla de tres líneas es la que hace que el resto del desafío sea posible: sin ella, el botón con `width: 100%` y `padding: 16px` del paso 5 se saldría de la tarjeta.

Paso 2 — La columna de lectura (requisito 2)

```
1  body {  
2    font-family: Arial, sans-serif;  
3    background: #f5f5f7;  
4    color: #1d1d1f;  
5    max-width: 620px;  
6    margin: 32px auto;  
7    padding: 0 16px;  
8  }  
9  
10 h1 {  
11   margin-bottom: 24px;  
12 }
```

Tres cosas pasan acá:

- `max-width: 620px` — "ancho máximo". En una pantalla angosta el body ocupa lo que hay; en una pantalla ancha, se planta en 620px. Es lo contrario de `width`, que fija el ancho siempre.
- `margin: 32px auto` — **32px arriba y abajo, auto a los lados**. El valor `auto` le dice al navegador "calculá vos": reparte todo el espacio sobrante en partes iguales a izquierda y derecha, y el bloque queda centrado. Es el truco de centrado de bloques en CSS.
- `padding: 0 16px` — un colchón lateral para que, en un celular, el texto no quede pegado al borde de la pantalla.

Por qué `margin: auto` a veces no centra

`margin: auto` solo centra si el elemento **tiene un ancho definido** (con `width` o `max-width`) y es un elemento de bloque. Sin ancho, el bloque ya ocupa el 100% disponible: no hay sobrante para repartir y no pasa nada. Si alguna vez `margin: auto` "no funciona", casi siempre falta el `max-width`.

Paso 3 — La tarjeta base (requisitos 3 y 4)

```

1  .plan {
2    background: white;
3    width: 100%;
4    max-width: 420px;
5    padding: 24px;
6    border: 1px solid #d2d2d7;
7    border-radius: 16px;
8    margin: 0 auto 24px;
9  }
10
11 .plan h2 {
12   margin: 0 0 16px 0;
13   font-size: 20px;
14 }

```

La combinación `width: 100%` + `max-width: 420px` es un patrón que vas a usar toda la vida: *estirate hasta donde puedas, pero no pases de acá*. En una ventana angosta la tarjeta se adapta; en una ancha se queda en 420px y no se deforma.

El `margin: 0 auto 24px` tiene tres valores: **arriba 0, a los lados auto, abajo 24px**. El `auto` centra cada tarjeta dentro del body; el 24px es la separación entre una tarjeta y la siguiente.

Y otra vez `.plan h2 { margin: 0 0 16px 0; }`: sin eso, el margen superior del `h2` se suma al padding de la tarjeta y el título queda flotando demasiado abajo.

Paso 4 — La lista (requisito 6)

```

1  .plan ul {
2    list-style: none;
3    margin: 0 0 24px 0;
4    padding: 0;
5  }
6
7  .plan li {
8    padding: 8px 0;
9    border-bottom: 1px solid #f0f0f2;
10 }

```

Las `<ul>` vienen del navegador con dos cosas que casi nunca queremos: las viñetas y un sangrado de unos 40px a la izquierda. `list-style: none` saca las primeras y `padding: 0` el segundo.

Después, `padding: 8px 0` en cada `<li>` le da aire arriba y abajo, y la línea de 1px separa un ítem del siguiente. Fijate que se usó **padding** y no **margin**: queremos que la línea divisoria quede a la distancia correcta del texto, y el padding es lo que empuja desde adentro.

Paso 5 — El botón (requisito 7)

```

1  .plan button {
2    width: 100%;
3    padding: 16px;
4    border: 0;
5    border-radius: 8px;
6    background: #e8f1fb;
7    color: #0057b8;
8    font: inherit;
9    font-weight: bold;
10   cursor: pointer;
11 }

```

Este es el momento donde se cobra el `border-box` del paso 1. `width: 100%` significa "todo el ancho **interno** de la tarjeta" y el `padding: 16px` se acomoda adentro de ese 100%. Sin `border-box`, el botón mediría 100% + 32px y sobresaldría de la tarjeta.

Dos detalles que separan un botón hecho de uno improvisado: `font: inherit` le hace heredar la tipografía de la página (los botones traen una tipografía propia del sistema, que desentona), y `cursor: pointer` hace que el mouse muestre la manito, que es la señal universal de "esto se puede clicar".

Paso 6 — El plan destacado (requisito 5)

```

1  .recomendado {
2    border-color: #0057b8;
3    background: #f7fbff;
4  }
5
6  .recomendado button {
7    background: #0057b8;
8    color: white;
9  }
10
11 .recomendado button:hover {
12   background: #003f88;
13 }

```

La decisión de diseño más importante del desafío

La tentación es hacer el plan recomendado más grande: más padding, un borde de 3px, una escala. Y ahí se rompe todo: las tres tarjetas dejan de estar alineadas y el conjunto se ve desprolijo.

La solución destaca **sin cambiar una sola medida**. Mismo padding, mismo `max-width`, mismo borde de 1px. Lo único que cambia es el **color** del borde y del fondo, y el relleno del botón. Se nota igual, y la grilla queda intacta. Esa es la diferencia entre destacar y desordenar.

Notá también que `.recomendado` escribe `border-color`, no `border`. Si escribiera `border: 1px solid #0057b8` estaría repitiendo el grosor y el estilo que `.plan` ya definió. Cambiar solo lo que cambia es lo que hace que el CSS sea fácil de mantener.

Solución completa

css/estilos.css – solución completa

CSS

```

1  /* =====
2     Desafío Clase 7 – Planes de servicio
3     css/estilos.css
4     -----
5     ESTRATEGIA
6     Los tres <article> comparten class="plan" y uno además tiene
7     .recomendado. Todo lo estructural se escribe una vez en .plan;
8     .recomendado solo cambia colores. Así los tres planes quedan
9     exactamente del mismo tamaño y el destacado no rompe la grilla.
10    Todos los espacios salen de la escala 8 / 16 / 24 / 32.
11    ===== */
12
13 /* ---- REQUISITO 1: border-box global ----- */
14 /* Con esto, width y height incluyen el padding y el borde.
15    Es la primera línea de casi cualquier CSS moderno. */
16 * {
17     box-sizing: border-box;
18 }
19
20 /* ---- REQUISITO 2: la columna de lectura ----- */
21 /* max-width le pone un techo al ancho; margin auto a los lados
22    reparte el sobrante en partes iguales y centra el bloque. */
23 body {
24     font-family: Arial, sans-serif;
25     background: #f5f5f7;
26     color: #1d1d1f;
27     max-width: 620px;
28     margin: 32px auto;
29     padding: 0 16px;
30 }
31
32 h1 {
33     margin-bottom: 24px;
34 }
35
36 /* ---- REQUISITOS 3 y 4: la regla base de los tres planes ---- */
37 /* width: 100% + max-width = ancho adaptable con tope.
38    margin: 0 auto 24px → 0 arriba, auto a los lados (centra),
39    24px abajo (la separación entre tarjetas). */
40 .plan {
41     background: white;
42     width: 100%;
43     max-width: 420px;
44     padding: 24px;
45     border: 1px solid #d2d2d7;
46     border-radius: 16px;
47     margin: 0 auto 24px;
48 }
49
50 /* margin-top 0 para que el título arranque justo donde
51    termina el padding de la caja. */
52 .plan h2 {
53     margin: 0 0 16px 0;
54     font-size: 20px;
55 }
56

```

```

57  /* ---- REQUISITO 6: la lista ----- */
58  /* Pierde las viñetas y el sangrado que trae por defecto el
59     navegador, y recibe un espacio propio de la escala. */
60  .plan ul {
61      list-style: none;
62      margin: 0 0 24px 0;
63      padding: 0;
64  }
65
66  .plan li {
67      padding: 8px 0;
68      border-bottom: 1px solid #f0f0f2;
69  }
70
71  /* ---- REQUISITO 7: el botón ----- */
72  /* Con border-box, width: 100% es el ancho interno: el botón
73     llega justo hasta donde empieza el padding de la tarjeta.
74     font: inherit hace que use la tipografía de la página y no
75     la que el navegador le pone por defecto a los formularios. */
76  .plan button {
77      width: 100%;
78      padding: 16px;
79      border: 0;
80      border-radius: 8px;
81      background: #e8f1fb;
82      color: #0057b8;
83      font: inherit;
84      font-weight: bold;
85      cursor: pointer;
86  }
87
88  /* ---- REQUISITO 5: el plan destacado ----- */
89  /* NO cambia de tamaño: mismo padding, mismo max-width, mismo
90     borde de 1px, así no desalinea la columna. Solo cambia el
91     color del borde y del fondo, y su botón se rellena para
92     leerse como la acción principal. */
93  .recomendado {
94      border-color: #0057b8;
95      background: #f7fbff;
96  }
97
98  .recomendado button {
99      background: #0057b8;
100     color: white;
101 }
102
103 .recomendado button:hover {
104     background: #003f88;
105 }

```

Cómo probarlo

1. Abrió `index.html`. Las tres tarjetas tienen que estar centradas y del mismo ancho.
2. Achicá la ventana del navegador de a poco. Las tarjetas se tienen que **angostar juntas**, sin que ninguna se desborde ni aparezca una barra de scroll horizontal.

- El botón tiene que terminar exactamente donde termina el texto de la lista de arriba. Si sobresale, falta el `border-box`.
- La tarjeta del medio se tiene que distinguir **sin ser más grande**: pasá el ojo por los bordes superiores de las tres, tienen que estar alineados en la misma columna.
- Apretá `F12`, hacé clic en una tarjeta y mirá el diagrama de cajas del panel derecho (el rectángulo de colores). Ahí ves gráficamente el `margin`, el `border`, el `padding` y el contenido. Es la mejor herramienta que existe para entender el Box Model.

Errores frecuentes

Qué pasa	Por qué	Cómo se arregla
El botón se sale de la tarjeta	Falta <code>box-sizing: border-box</code>	Agregá la regla <code>*</code> al principio del archivo.
Las tarjetas quedan pegadas a la izquierda	Falta <code>margin: 0 auto</code> en <code>.plan</code> , o falta el <code>max-width</code>	Necesitás las dos cosas: un ancho con tope y el <code>auto</code> a los lados.
La lista queda corrida a la derecha	No se anuló el <code>padding-left</code> por defecto de <code></code>	<code>padding: 0</code> en <code>.plan ul</code> .
El plan recomendado desalinea a los otros dos	Se le cambió el <code>padding</code> o el grosor del borde	Solo cambiale colores. Si necesitás un borde más grueso, usá <code>outline</code> o una <code>box-shadow</code> , que no ocupan espacio.
El botón usa una tipografía distinta al resto	Los controles de formulario no heredan la tipografía	<code>font: inherit</code> en el botón.

Para pensar antes de entregar

Recorré tu archivo y buscá cada número que sea un espacio (`padding` o `margin`). ¿Están todos en 8, 16, 24 o 32? Si aparece un 10, un 15 o un 20, preguntate qué pasaría si lo llevaras al valor de la escala más cercano. En la enorme mayoría de los casos: se ve mejor.

CAPÍTULO 3 · CLASE 8

Desafío: Tema automático y selector manual

Una página que arranca en el modo claro u oscuro que prefiere el sistema operativo, y que además se puede cambiar a mano con un botón. Es el primer proyecto con tres archivos: HTML, CSS y JavaScript.

Ficha del desafío

Clase	Clase 8 — Color, tipografía y variables CSS
Desafío	Tema automático y selector manual
Qué se practica	Variables CSS (custom properties), <code>var()</code> , selectores de atributo, <code>@media (prefers-color-scheme)</code> , especificidad y contraste
Archivos del proyecto	<code>index.html</code> + <code>css/estilos.css</code> + <code>js/tema.js</code>
Qué se toca	El CSS. El HTML y el JavaScript vienen dados.

Estructura del proyecto

Este es el primer desafío con las tres tecnologías, así que aparece la carpeta `js/`:

```
desafio-clase-08/  
├─ index.html  
├─ css/  
│   └─ estilos.css  
└─ js/  
    └─ tema.js
```

El archivo `index.html`

Acá hay **tres** diferencias respecto de lo que ves en la plataforma, y las tres importan:

1. El `<link>` a `css/estilos.css`, como siempre.
2. El `<script>` con la función `toggleTema()` que en la plataforma está escrito dentro del HTML, acá se mudó a su propio archivo `js/tema.js` y se lo trae con `<script src="js/tema.js"></script>`.

3. El atributo `data-tema="claro"` en la etiqueta `<html>` — que ya estaba, pero conviene mirarlo bien porque es la pieza central del mecanismo.

```

index.html HTML
1 <!DOCTYPE html>
2 <html lang="es" data-tema="claro">
3   <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Modo oscuro</title>
7
8     <!-- Tipografía externa: se descarga de Google Fonts.
9          Necesita conexión a internet para verse. -->
10    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
11          rel="stylesheet">
12
13    <!-- Nuestra hoja de estilos. Va DESPUÉS de la tipografía,
14          para poder usarla en las reglas. -->
15    <link rel="stylesheet" href="css/estilos.css">
16  </head>
17  <body>
18    <header>
19      <h1>Mi App</h1>
20      <button onclick="toggleTema()">🌙 Cambiar tema</button>
21    </header>
22
23    <main>
24      <section>
25        <h2>Bienvenido</h2>
26        <p>Esta página cambia de modo claro a oscuro usando variables CSS.</p>
27      </section>
28
29      <div class="card">
30        <h3>Tarjeta de ejemplo</h3>
31        <p>El fondo, el texto y el borde de esta tarjeta cambian con el tema.</p>
32        <button class="btn">Acción</button>
33      </div>
34
35      <div class="card">
36        <h3>Segunda tarjeta</h3>
37        <p>Todos los colores vienen de variables — no hay ningún color fijo en el CSS.</p>
38        <button class="btn">Otra acción</button>
39      </div>
40    </main>
41
42    <footer>
43      <p>©copy; 2025 Mi App</p>
44    </footer>
45
46    <!-- El JavaScript va al final del body, cuando el HTML ya existe. -->
47    <script src="js/tema.js"></script>
48  </body>
49 </html>

```

El archivo `js/tema.js`

Todo el JavaScript del proyecto son seis líneas. Vale la pena entenderlas porque muestran la división de trabajo entre CSS y JS:

`js/tema.js``JS`

```
1  /* =====
2    js/tema.js
3    Cambia el atributo data-tema del elemento <html> entre
4    "claro" y "oscuro". No toca ningún color: de eso se encarga
5    el CSS. El JavaScript solo mueve el interruptor.
6    ===== */
7
8  function toggleTema() {
9    // document.documentElement es el <html> de la página.
10   const html = document.documentElement;
11
12   // dataset.tema lee y escribe el atributo data-tema.
13   // Si está en "claro" lo pasa a "oscuro", y al revés.
14   html.dataset.tema = html.dataset.tema === "claro" ? "oscuro" : "claro";
15 }
```

La idea grande de este desafío

El JavaScript **no cambia ni un color**. Lo único que hace es escribir la palabra "oscuro" o "claro" en un atributo. Todos los colores están en el CSS, que reacciona a ese atributo.

Esa separación es una regla de oro del desarrollo web: **el JavaScript cambia el estado, el CSS decide cómo se ve cada estado**. Si mañana querés agregar un tema "sepia", no tocás una línea de JavaScript: agregás un bloque de variables al CSS.

Sobre el `onclick="toggleTema()"` del HTML: en proyectos reales se prefiere conectar los eventos desde el JavaScript (con `addEventListener`), pero eso es tema de las clases de JS. Acá el `onclick` alcanza y deja el mecanismo a la vista.

La consigna

CONSIGNA TAL COMO APARECE EN LA PLATAFORMA

OBJETIVO

Completar un sistema de temas que funcione automáticamente y también permita elegirlo con un botón.

DÓNDE TRABAJAR

Pestaña CSS. El botón y la función `toggleTema` ya existen en HTML.

REQUISITOS

1. Completá los siete tokens del tema claro.
2. Redefiní los mismos tokens en `[data-tema="oscuro"]`.

3. Completá @media (prefers-color-scheme: dark) para :root.
4. Usá únicamente variables en las reglas de componentes.
5. Elegí valores con contraste mínimo 4.5:1 para texto normal.
6. Probá primero la preferencia automática quitando temporalmente data-tema del elemento html.
7. Restaurá data-tema="claro" y comprobá cinco cambios seguidos con el botón.

ACLARACIÓN IMPORTANTE

El atributo data-tema representa una elección manual y sus reglas pueden prevalecer sobre el tema automático. La media query funciona como valor inicial cuando no existe esa elección.

CÓMO SABER QUE TERMINASTE

Fondo, superficies, textos, bordes y botones cambian juntos; nada queda con colores del tema anterior; las tarjetas se leen bien en ambos modos y no hay colores escritos directamente en sus reglas.

Punto de partida del CSS

A diferencia de los desafíos anteriores, acá el archivo viene armado: todos los componentes ya usan `var(--algo)`. Lo que falta son **los valores de esas variables**. Como están vacías, la página arranca sin colores.

css/estilos.css — punto de partida (recortado)

CSS

```

1  /* — MODO CLARO (por defecto) ————— */
2  :root, [data-tema="claro"] {
3      --bg-app: ;
4      /* fondo de la página */
5      --bg-superficie: ;
6      /* fondo de cards y header */
7      --color-texto: ;
8      /* texto principal */
9      --color-suave: ;
10     /* texto secundario */
11     --color-borde: ;
12     /* bordes */
13     --color-primario: ;
14     --color-btn-texto: ;
15 }
16
17 /* — MODO OSCURO ————— */
18 [data-tema="oscuro"] {
19     --bg-app: ;
20     --bg-superficie: ;
21     --color-texto: ;
22     --color-suave: ;
23     --color-borde: ;
24     --color-primario: ;
25     --color-btn-texto: ;
26 }
27
28 /* — DETECCIÓN AUTOMÁTICA DEL SISTEMA ————— */
29 @media (prefers-color-scheme: dark) {
30     :root {
31         /* Mismo que el modo oscuro de arriba */

```

```

32     }
33   }
34
35   /* — ESTILOS — SOLO VARIABLES, SIN COLORES FIJOS — */
36   /* (esta parte ya viene resuelta y no hay que tocarla) */

```

Cómo lo encara un desarrollador

Qué es un token de color

Una variable CSS se declara con dos guiones adelante (`--bg-app: #f5f5f7;`) y se usa con `var()` (`background-color: var(--bg-app);`). Pero lo importante no es la sintaxis: es **el nombre**.

Fijate que las variables no se llaman `--gris-claro` ni `--azul`, sino `--bg-app`, `--bg-superficie`, `--color-texto`. Están nombradas por **la función que cumplen**, no por el color que tienen. Eso es lo que permite que en modo oscuro `--bg-app` valga `#1c1c1c` sin que el nombre quede mintiendo. Una variable llamada `--gris-claro` con valor negro sería un chiste malo dentro de seis meses.

Las tres capas del sistema

Bloque	Qué representa	Cuándo gana
<code>:root</code>	El tema claro, el valor por defecto	Siempre, salvo que algo más específico lo pise
<code>@media (prefers-color-scheme: dark)</code>	Lo que prefiere el sistema operativo de quien visita	Si el sistema está en oscuro y no hay elección manual
<code>[data-tema="oscuro"]</code>	La elección manual , hecha con el botón	Siempre que exista, porque un selector de atributo pesa más que <code>:root</code>

El orden de la cascada acá no es casual: es exactamente el comportamiento que uno espera de una aplicación real. Arranca como prefiere tu sistema; si vos elegís otra cosa, tu elección manda.

Paso a paso

Paso 1 — Los siete tokens del tema claro (requisito 1)

```

1  :root, [data-tema="claro"] {
2    --bg-app: #f5f5f7;           /* fondo de la página */
3    --bg-superficie: #ffffff;     /* fondo de cards y header */
4    --color-texto: #1d1d1f;      /* texto principal */
5    --color-suave: #55555a;      /* texto secundario */
6    --color-borde: #e5e5ea;      /* bordes */
7    --color-primario: #0062cc;
8    --color-btn-texto: #ffffff;
9  }

```

Dos criterios detrás de estos valores:

- **El fondo de la app y el de las superficies no son el mismo.** La página es gris muy claro (`#f5f5f7`) y las tarjetas son blancas (`#ffffff`). Esa diferencia mínima es la que hace que las tarjetas se vean "apoyadas" sobre la página. Si los dos fueran blancos, las tarjetas desaparecerían.
- **El texto secundario no es gris claro, es gris medio.** `#55555a` sobre blanco da un contraste de aproximadamente 7:1. El mínimo que exigen las pautas de accesibilidad (WCAG) para texto normal es 4.5:1. Un `#999999` de moda daría 2.8:1 y sería ilegible para mucha gente.

Cómo se mide el contraste

Las DevTools lo hacen solas: hacé clic en el cuadradito de color que aparece al lado de una propiedad `color` en el panel de estilos y el selector de color te muestra el ratio de contraste con el fondo, con un tilde o una cruz. También hay calculadoras online buscando "contrast checker".

Paso 2 — Los mismos siete tokens en oscuro (requisito 2)

```

1  [data-tema="oscuro"] {
2    --bg-app: #1c1c1e;
3    --bg-superficie: #2c2c2e;
4    --color-texto: #f5f5f7;
5    --color-suave: #a9a9b0;
6    --color-borde: #38383a;
7    --color-primario: #0a84ff;
8    --color-btn-texto: #ffffff;
9  }

```

La clave es que son **exactamente los mismos siete nombres**. Si acá te olvidás uno, ese token conserva el valor del tema claro y vas a ver, por ejemplo, un borde blanco brillante en medio de una tarjeta oscura. Es el error más común del desafío.

Dos valores merecen explicación:

- `--color-suave` pasó de `#55555a` a `#a9a9b0`: se **aclaró**. En modo oscuro el texto secundario tiene que ser más claro que el fondo, no más oscuro. No alcanza con invertir la idea: hay que recalcular el contraste contra el

nuevo fondo.

- `--color-primario` pasó de `#0062cc` a `#0a84ff`: un azul más luminoso. Un azul medio que se ve perfecto sobre blanco pierde contraste sobre un fondo oscuro y queda apagado.

Y notá que el fondo oscuro es `#1c1c1e`, no `#000000`. El negro puro sobre una pantalla brillante produce un contraste tan alto que cansa la vista y hace que el texto blanco parezca "vibrar". Casi todos los sistemas operativos usan un gris muy oscuro.

Paso 3 — La detección automática (requisito 3)

```
1 @media (prefers-color-scheme: dark) {
2   :root {
3     --bg-app: #1c1c1e;
4     --bg-superficie: #2c2c2e;
5     --color-texto: #f5f5f7;
6     --color-suave: #a9a9b0;
7     --color-borde: #38383a;
8     --color-primario: #0a84ff;
9     --color-btn-texto: #ffffff;
10  }
11 }
```

`prefers-color-scheme` es una media query que no pregunta por el ancho de la pantalla sino por **una preferencia de la persona**, configurada en su sistema operativo (en Windows, Configuración → Personalización → Colores; en macOS, Preferencias → General; en el celular, en Pantalla). El navegador la reporta y el CSS puede reaccionar.

Sí, los valores están repetidos respecto del bloque anterior. Es una molestia real y hay formas de evitarla, pero requieren técnicas que exceden esta clase. Con lo visto hasta acá, repetir es la solución correcta y honesta.

Paso 4 — El botón que se olvida (requisito 4)

Este es el detalle que separa un desafío entregado de uno terminado. El botón "Cambiar tema" del header no tiene ninguna regla propia en el archivo de partida, así que usa los colores por defecto del navegador — que no cambian con el tema. Resultado: cambiás a oscuro y el botón queda gris claro, como una mancha.

```
1 header button {
2   background-color: var(--bg-superficie);
3   color: var(--color-texto);
4   border: 1px solid var(--color-borde);
5   border-radius: 8px;
6   padding: 8px 14px;
7   font-family: inherit;
8   font-size: 14px;
9   cursor: pointer;
10  transition: background-color 0.3s, border-color 0.3s, color 0.3s;
11 }
```

La consigna dice "nada queda con colores del tema anterior". Para verificarlo hay una técnica simple: cambiá a oscuro y **recorré la pantalla buscando cualquier cosa que siga clara**. Lo que encuentres, es lo que falta.

Sobre las transiciones

`transition: background-color 0.3s, color 0.3s;` hace que el cambio de color no sea un salto brusco sino un fundido de tres décimas de segundo. No es decoración: un cambio instantáneo de toda la pantalla es desconcertante; el fundido le da al ojo tiempo de seguir lo que pasó. Fíjate que está puesto en cada elemento que cambia de color: body, header, botones y tarjetas.

Solución completa

css/estilos.css – solución completa

CSS

```

1  /* =====
2  Desafío Clase 8 – Tema automático y selector manual
3  css/estilos.css
4  -----
5  CÓMO CONVIVEN LOS TRES BLOQUES DE TOKENS
6  1. :root → el tema claro, que es el valor por defecto.
7  2. [data-tema="oscuro"] → la elección MANUAL del botón.
8  3. @media (prefers-color-scheme: dark) → la preferencia del SISTEMA.
9
10 El orden y la especificidad importan. Un selector de atributo
11 ([data-tema="oscuro"]) pesa más que :root, así que la elección
12 manual siempre le gana a la automática. La media query actúa como
13 valor inicial cuando la persona todavía no eligió nada.
14 ===== */
15
16 /* — MODO CLARO (por defecto) ————— */
17 :root, [data-tema="claro"] {
18   --bg-app: #f5f5f7;
19   /* fondo de la página */
20   --bg-superficie: #ffffff;
21   /* fondo de cards y header */
22   --color-texto: #1d1d1f;
23   /* texto principal – contraste ~16:1 sobre blanco */
24   --color-suave: #55555a;
25   /* texto secundario – ~7:1, supera el mínimo de 4.5:1 */
26   --color-borde: #e5e5ea;
27   /* bordes */
28   --color-primario: #0062cc;
29   --color-btn-texto: #ffffff;
30 }
31
32 /* — MODO OSCURO ————— */
33 [data-tema="oscuro"] {
34   --bg-app: #1c1c1e;
35   --bg-superficie: #2c2c2e;
36   --color-texto: #f5f5f7;
37   --color-suave: #a9a9b0;
38   /* aclarado respecto del claro para mantener 4.5:1 sobre #2c2c2e */
39   --color-borde: #38383a;
40   --color-primario: #0a84ff;
41   /* más luminoso: sobre fondo oscuro un azul medio pierde contraste */
42   --color-btn-texto: #ffffff;
43 }
44
45 /* — DETECCIÓN AUTOMÁTICA DEL SISTEMA ————— */
46 @media (prefers-color-scheme: dark) {

```

```

47  :root {
48    /* Mismos valores que el modo oscuro manual.
49     * Este bloque es el que actúa si se quita data-tema del <html>. */
50    --bg-app: #1c1c1e;
51    --bg-superficie: #2c2c2e;
52    --color-texto: #f5f5f7;
53    --color-suave: #a9a9b0;
54    --color-borde: #38383a;
55    --color-primario: #0a84ff;
56    --color-btn-texto: #ffffff;
57  }
58 }
59
60 /* — ESTILOS — SOLO VARIABLES, SIN COLORES FIJOS ———— */
61 *, *::before, *::after {
62   box-sizing: border-box;
63   margin: 0;
64   padding: 0;
65 }
66
67 body {
68   font-family: "Inter", sans-serif;
69   background-color: var(--bg-app);
70   color: var(--color-texto);
71   transition: background-color 0.3s, color 0.3s;
72   min-height: 100vh;
73 }
74
75 header {
76   background-color: var(--bg-superficie);
77   border-bottom: 1px solid var(--color-borde);
78   padding: 16px 32px;
79   display: flex;
80   justify-content: space-between;
81   align-items: center;
82 }
83
84 /* El botón que cambia el tema también tiene que respetar el tema,
85  * si no queda con los colores del modo anterior. */
86 header button {
87   background-color: var(--bg-superficie);
88   color: var(--color-texto);
89   border: 1px solid var(--color-borde);
90   border-radius: 8px;
91   padding: 8px 14px;
92   font-family: inherit;
93   font-size: 14px;
94   cursor: pointer;
95   transition: background-color 0.3s, border-color 0.3s, color 0.3s;
96 }
97
98 main {
99   max-width: 800px;
100  margin: 0 auto;
101  padding: 40px 20px;
102 }
103
104 section {
105   margin-bottom: 32px;
106 }
107

```

```
108 h2 {
109   margin-bottom: 12px;
110 }
111
112 p {
113   line-height: 1.7;
114   color: var(--color-suave);
115 }
116
117 .card {
118   background-color: var(--bg-superficie);
119   border: 1px solid var(--color-borde);
120   border-radius: 12px;
121   padding: 24px;
122   margin-bottom: 16px;
123   transition: background-color 0.3s, border-color 0.3s;
124 }
125
126 .card h3 {
127   margin-bottom: 8px;
128 }
129
130 .btn {
131   margin-top: 16px;
132   padding: 10px 20px;
133   background-color: var(--color-primario);
134   color: var(--color-btn-texto);
135   border: none;
136   border-radius: 8px;
137   font-family: inherit;
138   font-size: 14px;
139   cursor: pointer;
140   transition: background-color 0.3s;
141 }
142
143 footer {
144   text-align: center;
145   padding: 24px;
146   color: var(--color-suave);
147   font-size: 0.875rem;
148   border-top: 1px solid var(--color-borde);
149 }
```

Cómo probarlo

Este desafío tiene dos mecanismos distintos y hay que probar los dos por separado.

Prueba A — El botón (elección manual)

1. Abrí `index.html`. Tiene que verse en claro (porque el HTML dice `data-tema="claro"`).
2. Hacé clic en "🌙 Cambiar tema" **cinco veces seguidas**. Cada clic tiene que cambiar el tema completo, sin que nada quede a mitad de camino.
3. Con las DevTools abiertas en la pestaña Elements, mirá la etiqueta `<html>` mientras hacés clic: vas a ver el atributo `data-tema` cambiando de valor en vivo. Eso es todo el JavaScript trabajando.

Prueba B — La preferencia del sistema (automática)

1. En las DevTools, apretá `Ctrl+Shift+P` (o `Cmd+Shift+P`), escribí `"dark"` y elegí **Emulate CSS prefers-color-scheme: dark**. Es la forma rápida de simular un sistema en modo oscuro sin cambiar la configuración de tu computadora.
2. Ahora, en la pestaña Elements, hacé clic derecho sobre `<html>` y borra el atributo `data-tema`. La página tiene que pasar a oscuro sola.
3. Volvé a poner `data-tema="claro"`: la página tiene que volver a claro **aunque el sistema esté en oscuro**. Eso demuestra que la elección manual le gana a la automática, que es justo lo que pide la aclaración de la consigna.

Errores frecuentes

Qué pasa	Por qué	Cómo se arregla
La página se ve sin colores, todo blanco y negro	Alguna variable quedó vacía. <code>var(--x)</code> de una variable sin valor no aplica nada	Revisá que las siete tengan valor en los tres bloques.
En oscuro queda un borde o un texto claro suelto	Falta redefinir ese token en <code>[data-tema="oscuro"]</code>	Compará los dos bloques nombre por nombre: tienen que ser los mismos siete.
El botón "Cambiar tema" no cambia de color	No tiene reglas propias y usa los colores por defecto del navegador	Agregá la regla <code>header button</code> con variables.
El botón no hace nada al hacer clic	Falta <code></script></code> , o la ruta <code>js/tema.js</code> está mal	Mirá la pestaña Console de las DevTools: si dice <code>toggleTema is not defined</code> o <code>404</code> , es la ruta.
La tipografía se ve como Times New Roman	No hay internet, o el <code><link></code> de Google Fonts está mal	Es solo estético: el resto funciona igual. La página cae en la <code>sans-serif</code> de respaldo.
El modo oscuro se ve "sucio", con textos difíciles de leer	Se invirtieron los colores mecánicamente sin recalcular contrastes	Los grises intermedios hay que aclararlos, y el azul principal subirlo de luminosidad.

Para pensar antes de entregar

Buscá en tu archivo, de la sección de componentes para abajo, cualquier valor de color escrito a mano: un `#`, un `rgb()`, o una palabra como `white`. No debería haber ninguno. Todos los colores tienen que estar arriba, en los bloques de tokens, y los componentes solo deberían decir `var(--algo)`. Si lo lograste, agregar un tercer tema te costaría siete líneas.

CAPÍTULO 4 · CLASE 9

Desafío: Organizar una aplicación de música

Barra lateral, catálogo de álbumes y reproductor abajo: la estructura de cualquier aplicación de música. Cuatro contenedores Flexbox y ninguna propiedad nueva más que las de la clase.

Ficha del desafío

Clase	Clase 9 — Flexbox desde cero: ordenar y alinear elementos
Desafío	Organizar una aplicación de música
Qué se practica	<code>display: flex</code> , <code>flex-direction</code> , <code>justify-content</code> , <code>align-items</code> , <code>flex-wrap</code> , <code>gap</code> y la propiedad abreviada <code>flex</code>
Archivos del proyecto	<code>index.html</code> + <code>css/estilos.css</code>
Qué se toca	Solo las cuatro zonas marcadas como BLOQUE en el CSS.

Estructura del proyecto

```
desafio-clase-09/  
├─ index.html  
└─ css/  
    └─ estilos.css
```

¿Y la carpeta js/?

No hace falta. Este desafío se resuelve entero con HTML y CSS. La plataforma tiene una pestaña JS siempre visible, pero en un proyecto real **no se crean archivos que no se usan**: si no hay JavaScript, no hay carpeta `js/` ni etiqueta `<script>`.

El archivo `index.html`

Antes de leer el CSS, conviene leer el HTML como si fuera un plano. Fijate en el **anidamiento**: quién está adentro de quién es lo que va a determinar qué elemento puede alinear a qué.

index.html

HTML

```

1  <!DOCTYPE html>
2  <html lang="es">
3    <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>MiMusic</title>
7      <link rel="stylesheet" href="css/estilos.css">
8    </head>
9    <body>
10     <div class="pantalla">
11
12       <header class="cabecera">
13         <strong>♪ MiMusic</strong>
14       </header>
15
16       <div class="app">
17         <aside class="sidebar">
18           <nav class="sidebar__nav">
19             <a href="#">Inicio</a>
20             <a href="#">Buscar</a>
21             <a href="#">Favoritos</a>
22           </nav>
23         </aside>
24
25         <main class="contenido">
26           <h1>Álbumes recomendados</h1>
27           <section class="albumes">
28             <article class="album">
29               <div class="tapa tapa--1"></div>
30               <strong>Horizonte</strong>
31               <span>Varios artistas</span>
32             </article>
33             <article class="album">
34               <div class="tapa tapa--2"></div>
35               <strong>Concentración</strong>
36               <span>Lista de estudio</span>
37             </article>
38             <article class="album">
39               <div class="tapa tapa--3"></div>
40               <strong>Clásicos</strong>
41               <span>Grandes éxitos</span>
42             </article>
43           </section>
44         </main>
45       </div>
46
47       <footer class="player">
48         <div>
49           <strong>Horizonte</strong>
50           <span>Las Luces</span>
51         </div>
52         <div class="player__controles">
53           <button>◀</button>
54           <button>▶</button>
55           <button>▶</button>
56         </div>
57         <span>Volumen 70%</span>
58       </footer>
59

```

```

60     </div>
61   </body>
62 </html>

```

Traducido a palabras, la estructura dice:

```

.pantalla          ← columna: cabecera, app y reproductor
├── .cabecera      ← el título "♪ MiMusic"
├── .app           ← fila: barra lateral + contenido
│   ├── .sidebar
│   │   ├── .sidebar__nav ← columna: los tres enlaces
│   │   └── .contenido
│   │       ├── .albumes ← fila con wrap: las tres tarjetas
│   └── .player     ← fila: info, controles, volumen
│       └── .player__controles ← fila: los tres botones

```

Sobre los nombres con doble guión bajo

`sidebar__nav` y `player__controles` usan una convención llamada **BEM**: el doble guión bajo separa el *bloque* de su *elemento interno*. Se lee "el nav que pertenece a la sidebar". No es obligatorio, pero cuando el proyecto crece, un nombre así te dice de dónde viene la clase sin tener que buscar en el HTML. Lo mismo hace `tapa--1` con doble guión medio: ahí marca una *variante* del mismo elemento.

La consigna

CONSIGNA TAL COMO APARECE EN LA PLATAFORMA

OBJETIVO

Usar lo aprendido en los cinco ejercicios para organizar una aplicación de música. No necesitás ninguna propiedad nueva.

QUÉ VIENE PREPARADO

El HTML, los colores, los tamaños y el aspecto de cada componente. Al comenzar, las secciones aparecen una debajo de otra.

RESULTADO QUE TENÉS QUE CONSTRUIR

MiMusic	
Inicio	Álbumes
Buscar	[A] [B] [C]
Favoritos	
Canción [◀] [▶] [▶] Volumen	

DÓNDE ESCRIBIR

Pestaña CSS. Completá únicamente las reglas marcadas como BLOQUE 1, 2, 3 y 4.

BLOQUE 1 – CUERPO DE LA APLICACIÓN

En `.app` activá Flexbox. La barra lateral y el contenido deben quedar en una fila. Agregá `flex: 1` para que esta zona use el espacio disponible.

BLOQUE 2 – BARRA LATERAL Y CONTENIDO

En `.sidebar` usá `flex: 0 0 130px`. Esto mantiene un ancho de 130px.

En `.contenido` usá `flex: 1` para que ocupe el resto.

En `.sidebar__nav` activá Flexbox, elegí dirección `column` y agregá `gap: 8px`.

BLOQUE 3 – ÁLBUMES

En `.albumes` activá Flexbox, permití nuevas filas con `wrap` y agregá `gap: 12px`.

En `.album` usá `flex: 1 1 120px`.

BLOQUE 4 – REPRODUCTOR

En `.player` activá Flexbox, separá los extremos con `space-between`, centrá verticalmente y agregá `gap: 12px`.

En `.player__controles` activá Flexbox y agregá `gap: 12px`.

CÓMO COMPROBARLO

- Barra lateral y contenido están uno al lado del otro.
- Los enlaces de la barra lateral están uno debajo de otro.
- Los álbumes forman filas y bajan cuando no entran.
- El reproductor tiene información, controles y volumen en una sola fila.
- No aparece desbordamiento horizontal.

ANTES DE MARCAR COMO COMPLETADO

Explicá con tus palabras qué elemento es el contenedor Flexbox en cada bloque.

Si podés identificar los cuatro contenedores, entendiste el desafío.

Punto de partida del CSS

El archivo viene con todos los colores y tamaños ya resueltos. Lo único vacío son los cuatro bloques marcados. Al abrir la página, todo aparece apilado uno debajo del otro: así se ve una página sin Flexbox.

css/estilos.css – punto de partida (recortado)

CSS

```
1  :root {
2    --fondo: #181818;
3    --panel: #282828;
4    --texto: #f5f5f7;
5    --suave: #b3b3b3;
6    --acento: #1db954;
7  }
8  * {
9    box-sizing: border-box;
10 }
11 body {
12   margin: 0;
13   font-family: Arial, sans-serif;
14   background: var(--fondo);
15   color: var(--texto);
16 }
17 .pantalla {
18   min-height: 100vh;
```

```
19   display: flex;
20   flex-direction: column;
21 }
22 .cabecera {
23   padding: 16px 20px;
24   border-bottom: 1px solid #444;
25 }
26 .app {
27   /* BLOQUE 1 */
28 }
29 .sidebar {
30   padding: 18px 12px;
31   background: #121212;
32   /* BLOQUE 2 */
33 }
34 .sidebar__nav {
35   /* BLOQUE 2 */
36 }
37 .contenido {
38   padding: 20px;
39   /* BLOQUE 2 */
40 }
41 .albumes {
42   /* BLOQUE 3 */
43 }
44 .album {
45   padding: 12px;
46   border-radius: 8px;
47   background: var(--panel);
48   /* BLOQUE 3 */
49 }
50 .player {
51   padding: 14px 18px;
52   border-top: 1px solid #444;
53   background: var(--panel);
54   /* BLOQUE 4 */
55 }
56 .player__controles {
57   /* BLOQUE 4 */
58 }
```

Cómo lo encara un desarrollador

La única pregunta que importa: ¿padre o hijo?

Todo Flexbox se reduce a entender esto. Hay **dos familias de propiedades** y van en lugares distintos:

Van en el contenedor (el padre)	Van en los ítems (los hijos)
<code>display: flex</code> <code>flex-direction</code> <code>justify-content</code> <code>align-items</code> <code>flex-wrap</code> <code>gap</code>	<code>flex</code> (y sus partes: <code>flex-grow</code> , <code>flex-shrink</code> , <code>flex-basis</code>) <code>align-self</code> <code>order</code>

Escribir `justify-content` en un hijo no hace nada. Escribir `flex: 1` en un elemento cuyo padre no es flex, tampoco. **El 90% de los problemas con Flexbox son una propiedad puesta en el elemento equivocado.**

Por eso el primer paso del desafío no es escribir CSS: es mirar el HTML y hacer la lista de los contenedores. En este caso son cuatro (cinco contando el anidado), y están todos identificados arriba, en el árbol de la estructura.

Cómo leer `flex: 1 1 120px`

La propiedad `flex` es la abreviatura de tres cosas, siempre en este orden:

Posición	Nombre real	Qué significa
1°	<code>flex-grow</code>	¿Cuánto crece si sobra espacio? 0 = nada, 1 = todo lo que pueda.
2°	<code>flex-shrink</code>	¿Cuánto se achica si falta espacio? 0 = nada, 1 = lo que haga falta.
3°	<code>flex-basis</code>	¿De qué tamaño parte ? Puede ser una medida o <code>auto</code> .

- `flex: 0 0 130px` → "no crezcas, no te achiques, medí 130px". Es una **columna fija**.
- `flex: 1` → atajo de `1 1 0`: "crecé todo lo que puedas". Es una **columna elástica** que se come el espacio sobrante.
- `flex: 1 1 120px` → "apuntá a 120px, pero crecé o achicate según haga falta". Es la receta de las **tarjetas adaptables**.

Paso a paso

BLOQUE 1 — El cuerpo de la aplicación

```

1  .app {
2    display: flex; /* barra lateral y contenido, lado a lado */
3    flex: 1;      /* ocupa toda la altura que sobra */
4  }
```

Dos líneas que hacen cosas **distintas**, y esto es lo más fino del desafío:

- `display: flex` actúa hacia **adentro**: convierte a `.app` en contenedor y pone a sus hijos (`.sidebar` y `.contenido`) en fila.
- `flex: 1` actúa hacia **afuera**: `.app` es a su vez hijo de `.pantalla`, que ya es un flex en columna. Este `1` le dice "quedate con toda la altura que sobre entre la cabecera y el reproductor".

Un mismo elemento puede ser contenedor y ítem al mismo tiempo. Si sacás el `flex: 1`, la app mide solo lo que miden sus contenidos y el reproductor queda flotando en el medio de la pantalla en vez de pegado abajo. Probalo: es la forma más rápida de entenderlo.

BLOQUE 2 — Barra lateral y contenido

```

1  .sidebar {
2    padding: 18px 12px;
3    background: #121212;
4    flex: 0 0 130px; /* no crece, no se achica, mide 130px */
5  }
6
7  .sidebar__nav {
8    display: flex;
9    flex-direction: column;
10   gap: 8px;
11 }
12
13 .contenido {
14   padding: 20px;
15   flex: 1; /* se queda con todo el resto del ancho */
16 }

```

Este es el patrón "**columna fija + columna elástica**", probablemente el más usado de toda la maquetación web. Un elemento con `flex: 0 0 medida` y su hermano con `flex: 1`. Lo vas a reconocer en el 90% de las aplicaciones que uses.

Y en el medio, `.sidebar__nav` hace algo distinto: se convierte en contenedor **propio**, con `flex-direction: column` para que los tres enlaces vayan uno debajo del otro con 8px de separación.

Cuando ponés `column`, los ejes se dan vuelta

En `row` (el valor por defecto), `justify-content` alinea en **horizontal** y `align-items` en **vertical**. Con `flex-direction: column` se invierten: `justify-content` pasa a ser vertical y `align-items`, horizontal.

La regla que nunca falla: `justify-content` trabaja sobre el **eje principal** (la dirección en la que se acomodan los ítems) y `align-items` sobre el **eje cruzado** (el perpendicular). Memorizá eso y no memorices "horizontal/vertical".

BLOQUE 3 — Los álbumes

```

1  .albumes {
2    display: flex;
3    flex-wrap: wrap;
4    gap: 12px;
5  }
6
7  .album {
8    padding: 12px;
9    border-radius: 8px;
10   background: var(--panel);
11   flex: 1 1 120px; /* apunta a 120px, pero crece y se achica */
12 }

```

Tres propiedades que trabajan juntas:

- `display: flex` pone las tarjetas en fila.
- `flex-wrap: wrap` autoriza a que **bajen de renglón** cuando no entran. Sin esto, Flexbox aprieta las tarjetas hasta deformarlas antes que dejarlas bajar. Es la diferencia entre una galería que se adapta y una que se rompe.
- `gap: 12px` separa las tarjetas entre sí, tanto en horizontal como en vertical, **sin agregar margen en los extremos**. Antes de que existiera `gap` esto se hacía con márgenes y restas; hoy es una línea.

Y en las tarjetas, `flex: 1 1 120px`: parten de 120px, pero si sobra espacio se reparten el sobrante en partes iguales (por eso las tres quedan del mismo ancho) y si falta, se achican hasta que el `wrap` las manda a la fila siguiente.

BLOQUE 4 — El reproductor

```

1  .player {
2    padding: 14px 18px;
3    border-top: 1px solid #444;
4    background: var(--panel);
5    display: flex;
6    justify-content: space-between;
7    align-items: center;
8    gap: 12px;
9  }
10
11 .player__controles {
12   display: flex;
13   gap: 12px;
14 }

```

El reproductor tiene tres hijos: la información de la canción, los controles y el volumen. `justify-content: space-between` hace exactamente lo que dice: manda el primero al extremo izquierdo, el último al derecho y reparte el espacio sobrante entre medio. Es la forma estándar de armar una barra con cosas en los extremos.

`align-items: center` los alinea verticalmente en el centro: sin eso, los tres se estiran a la altura del más alto y el texto queda arriba, desalineado con los botones.

Y `.player__controles` es el **contenedor anidado**: es hijo del player (y por eso el `space-between` del player lo trata como una sola pieza) y a la vez padre de los tres botones, que alinea en fila con 12px entre sí.

Solución completa

css/estilos.css – solución completa

CSS

```

1  /* =====
2  Desafío Clase 9 – MiMusic
3  css/estilos.css
4  -----
5  LOS CUATRO CONTENEDORES FLEX DE ESTE DESAFÍO
6  (esto es lo que pide explicar el "antes de marcar como completado")
7
8  BLOQUE 1 → .app           contiene la barra lateral y el contenido
9  BLOQUE 2 → .sidebar__nav contiene los tres enlaces del menú
10 BLOQUE 3 → .albumes       contiene las tres tarjetas de álbum
11 BLOQUE 4 → .player        contiene info, controles y volumen
12                      (y .player__controles, que contiene los tres botones)
13
14 .sidebar y .contenido NO son contenedores: son HIJOS de .app, y por
15 eso reciben flex: en vez de display: flex. Esa es la distinción
16 clave – display: flex se le pone al padre; flex: se le pone al hijo.
17 ===== */
18
19 :root {
20   --fondo: #181818;
21   --panel: #282828;
22   --texto: #f5f5f7;
23   --suave: #b3b3b3;
24   --acento: #1db954;
25 }
26
27 * {
28   box-sizing: border-box;
29 }
30
31 body {
32   margin: 0;
33   font-family: Arial, sans-serif;
34   background: var(--fondo);
35   color: var(--texto);
36 }
37
38 /* Ya venía resuelto: apila cabecera, app y reproductor en columna
39 y ocupa toda la altura de la ventana. */
40 .pantalla {
41   min-height: 100vh;
42   display: flex;
43   flex-direction: column;
44 }
45
46 .cabecera {
47   padding: 16px 20px;
48   border-bottom: 1px solid #444;
49 }
50
51 /* ---- BLOQUE 1 ----- */
52 /* display: flex → barra lateral y contenido quedan lado a lado.
53    flex: 1       → como .app es hijo de .pantalla (que está en
54                   columna), este 1 le hace ocupar toda la altura
55                   que sobra entre la cabecera y el reproductor.
56                   Sin esto, la app solo mediría lo que mide su

```

```

57         contenido y el reproductor quedaría flotando
58         en el medio de la pantalla. */
59     .app {
60         display: flex;
61         flex: 1;
62     }
63
64     /* ---- BLOQUE 2 ----- */
65     /* flex: 0 0 130px se lee:
66         0      no crece
67         0      no se achica
68         130px ese es su ancho, y punto.
69         Es la forma de fijar una columna: el ancho no se negocia. */
70     .sidebar {
71         padding: 18px 12px;
72         background: #121212;
73         flex: 0 0 130px;
74     }
75
76     /* Los enlaces van uno DEBAJO del otro, así que se cambia la
77         dirección del eje principal a column. Ojo: con column,
78         justify-content pasa a controlar el eje vertical y
79         align-items el horizontal – los ejes se dan vuelta. */
80     .sidebar__nav {
81         display: flex;
82         flex-direction: column;
83         gap: 8px;
84     }
85
86     .sidebar a {
87         padding: 8px;
88         border-radius: 6px;
89         color: var(--suave);
90         text-decoration: none;
91     }
92
93     .sidebar a:hover {
94         color: white;
95         background: #333;
96     }
97
98     /* El complemento de la barra lateral: la sidebar tiene ancho fijo
99         y el contenido se queda con todo el resto. */
100    .contenido {
101        padding: 20px;
102        flex: 1;
103    }
104
105    .contenido h1 {
106        margin-top: 0;
107        font-size: 22px;
108    }
109
110    /* ---- BLOQUE 3 ----- */
111    /* Mismo patrón del ejercicio 4: fila + wrap + gap. */
112    .albumes {
113        display: flex;
114        flex-wrap: wrap;
115        gap: 12px;
116    }
117

```

```

118  /* Apuntan a 120px pero se estiran o se encogen, y cuando no
119     entran bajan de fila gracias al wrap de arriba. */
120  .album {
121    padding: 12px;
122    border-radius: 8px;
123    background: var(--panel);
124    flex: 1 1 120px;
125  }
126
127  .album strong, .album span {
128    display: block;
129  }
130
131  .album span {
132    margin-top: 4px;
133    color: var(--suave);
134    font-size: 12px;
135  }
136
137  .tapa {
138    height: 90px;
139    margin-bottom: 10px;
140    border-radius: 6px;
141  }
142
143  .tapa--1 { background: linear-gradient(135deg, #0057b8, #68a7e8); }
144  .tapa--2 { background: linear-gradient(135deg, #6f42c1, #c79cff); }
145  .tapa--3 { background: linear-gradient(135deg, #b42318, #ff8a7f); }
146
147  /* ---- BLOQUE 4 ----- */
148  /* Tres hijos: info de la canción, controles y volumen.
149     space-between manda el primero a la izquierda, el último a la
150     derecha y deja los controles en el medio.
151     align-items: center los alinea a la misma altura. */
152  .player {
153    padding: 14px 18px;
154    border-top: 1px solid #444;
155    background: var(--panel);
156    display: flex;
157    justify-content: space-between;
158    align-items: center;
159    gap: 12px;
160  }
161
162  .player > div:first-child strong,
163  .player > div:first-child span {
164    display: block;
165  }
166
167  .player > div:first-child span {
168    color: var(--suave);
169    font-size: 12px;
170  }
171
172  /* Contenedor anidado: es hijo del .player y a la vez padre de
173     los tres botones. */
174  .player__controles {
175    display: flex;
176    gap: 12px;
177  }
178

```

```

179 .player button {
180   width: 34px;
181   height: 34px;
182   border: 0;
183   border-radius: 50%;
184   cursor: pointer;
185 }

```

Cómo probarlo

1. La barra lateral y el contenido tienen que estar **lado a lado**, no uno debajo del otro.
2. Los tres enlaces del menú, uno **debajo** del otro.
3. El reproductor tiene que quedar pegado al fondo de la ventana, aunque haya poco contenido.
4. Achicá la ventana de a poco: los álbumes se tienen que ir angostando y, en algún punto, **bajar de fila**. Nunca tiene que aparecer una barra de scroll horizontal.
5. En las DevTools, buscá en el panel de estilos el ícono chiquito que dice `flex` al lado de `display: flex` y hacé clic: el navegador **dibuja las líneas del contenedor sobre la página** y podés probar las alineaciones con botones. Es la mejor forma de aprender Flexbox.

La pregunta que cierra el desafío

La consigna pide algo que no es código: **explicar con tus palabras cuál es el contenedor Flexbox de cada bloque**. La respuesta es `.app`, `.sidebar__nav`, `.albumes` y `.player` (más `.player__controles` anidado adentro).

Si además podés explicar por qué `.sidebar` y `.contenido` **no** son contenedores —porque son hijos de `.app` y por eso reciben `flex`: y no `display: flex` — entonces entendiste Flexbox, no solo este desafío.

Errores frecuentes

Qué pasa	Por qué	Cómo se arregla
Todo sigue apilado uno debajo del otro	Falta <code>display: flex</code> en el contenedor	Sin esa línea, ninguna otra propiedad de Flexbox hace nada.
El reproductor queda en el medio de la pantalla	Falta <code>flex: 1</code> en <code>.app</code>	La app tiene que estirarse para empujar al player hacia abajo.
La barra lateral cambia de ancho al cambiar el contenido	Se usó <code>width: 130px</code> en vez de <code>flex: 0 0 130px</code>	Con <code>width</code> a secas, Flexbox igual puede achicarla. El <code>0</code> del medio (<code>flex-shrink</code>) es el que lo impide.
Los álbumes se aplastan y no bajan de fila	Falta <code>flex-wrap: wrap</code>	Por defecto Flexbox es <code>nowrap</code> : prefiere deformar antes que bajar.

Qué pasa	Por qué	Cómo se arregla
Los enlaces del menú quedan en fila	Falta <code>flex-direction: column</code> en <code>.sidebar_nav</code>	El valor por defecto es <code>row</code> .
<code>justify-content</code> no hace nada	Está escrito en un hijo en vez de en el contenedor	Va siempre en el elemento que tiene <code>display: flex</code> .

CAPÍTULO 5 · CLASE 10

Desafío: Organizar un panel de proyectos

La estructura clásica de un panel de administración —cabecera arriba, menú a la izquierda, contenido a la derecha— dibujada con áreas de Grid. Y adentro, una grilla de tarjetas que se reacomoda sola sin una sola media query.

Ficha del desafío

Clase	Clase 10 — CSS Grid desde cero: organizar filas y columnas
Desafío	Organizar un panel de proyectos
Qué se practica	<code>display: grid</code> , <code>grid-template-columns</code> , <code>grid-template-areas</code> , <code>grid-area</code> , la unidad <code>fr</code> , <code>repeat()</code> , <code>auto-fit</code> , <code>minmax()</code> y <code>span</code>
Archivos del proyecto	<code>index.html</code> + <code>css/estilos.css</code>
Qué se toca	Solo los cuatro bloques marcados.

Estructura del proyecto

```
desafio-clase-10/
├─ index.html
├─ css/
│   └─ estilos.css
```

El archivo `index.html`

index.html	HTML
<pre> 1 <!DOCTYPE html> 2 <html lang="es"> 3 <head> 4 <meta charset="UTF-8"> 5 <meta name="viewport" content="width=device-width, initial-scale=1.0"> 6 <title>Panel</title> 7 <link rel="stylesheet" href="css/estilos.css"> 8 </head> 9 <body> 10 <div class="panel"></pre>	

```

11
12     <header class="cabecera">Panel de proyectos</header>
13
14     <aside class="menu">
15         Inicio
16         <br>
17         Proyectos
18         <br>
19         Equipo
20     </aside>
21
22     <main class="contenido">
23         <h1>Estado general</h1>
24         <section class="tarjetas">
25             <article class="destacada">
26                 <strong>Resumen</strong>
27                 <p>12 proyectos activos</p>
28             </article>
29             <article>
30                 <strong>Pendientes</strong>
31                 <p>8 tareas</p>
32             </article>
33             <article>
34                 <strong>Equipo</strong>
35                 <p>6 personas</p>
36             </article>
37             <article>
38                 <strong>Entregas</strong>
39                 <p>3 esta semana</p>
40             </article>
41         </section>
42     </main>
43
44 </div>
45 </body>
46 </html>

```

Mirá el orden del HTML

En el HTML el orden es: cabecera, menú, contenido. Con Grid, **ese orden no determina dónde se ve cada cosa**: el CSS puede ubicarlos en cualquier celda. Es la gran diferencia con Flexbox, donde los ítems siguen el orden del documento.

Ojo: eso no significa que el orden del HTML dé igual. Un lector de pantalla y la navegación por teclado siguen el orden del **documento**, no el visual. Por eso el HTML tiene que estar en un orden lógico igual, y Grid se usa para acomodar, no para arreglar un HTML desordenado.

La consigna

CONSIGNA TAL COMO APARECE EN LA PLATAFORMA

OBJETIVO

Construir un panel usando solamente lo aprendido en esta clase.

RESULTADO ESPERADO

Cabecera arriba. Debajo, un menú angosto y el contenido. Dentro del contenido, tarjetas automáticas. La tarjeta Resumen ocupa dos columnas.

BLOQUE 1 – PÁGINA

En `.panel` activá Grid, creá columnas de 120px y 1fr, y dibujá estas áreas:

```
"cabecera cabecera"
"menu contenido"
```

BLOQUE 2 – ASIGNAR ÁREAS

Asigná cabecera, menu y contenido con `grid-area`.

BLOQUE 3 – TARJETAS

En `.tarjetas` activá Grid. Usá `repeat(auto-fit, minmax(120px, 1fr))` y gap de 12px.

BLOQUE 4 – DESTACADA

Hacé que `.destacada` ocupe dos columnas con `span 2`.

CÓMO COMPROBARLO

- El dibujo de áreas coincide con la pantalla.
- Las tarjetas cambian de cantidad por fila según el ancho.
- Resumen ocupa dos columnas.
- No usaste Flexbox, position ni media queries.

Punto de partida del CSS

css/estilos.css – punto de partida (recortado)

CSS

```
1  :root {
2    --oscuro: #1d1d1f;
3    --fondo: #f5f5f7;
4    --azul: #0057b8;
5  }
6  * {
7    box-sizing: border-box;
8  }
9  body {
10   margin: 0;
11   font-family: Arial, sans-serif;
12   background: var(--fondo);
13 }
14 .panel {
15   min-height: 100vh;
16   /* BLOQUE 1 */
17 }
18 .cabecera {
19   padding: 18px;
20   background: var(--oscuro);
21   color: white;
22   /* BLOQUE 2 */
23 }
24 .menu {
25   padding: 18px;
26   line-height: 2;
27   background: #e8f1fb;
28   /* BLOQUE 2 */
```

```

29 }
30 .contenido {
31   padding: 20px;
32   /* BLOQUE 2 */
33 }
34 .tarjetas {
35   /* BLOQUE 3 */
36 }
37 .tarjetas article {
38   padding: 18px;
39   border: 1px solid #d2d2d7;
40   border-radius: 8px;
41   background: white;
42 }
43 .destacada {
44   border-color: var(--azul) !important;
45   /* BLOQUE 4 */
46 }

```

Cómo lo encara un desarrollador

Grid se piensa dibujando

Flexbox acomoda cosas **en una dirección**: una fila o una columna. Grid trabaja con **filas y columnas a la vez**, y por eso permite algo que Flexbox no: dibujar el plano de la página con palabras.

La técnica es literal. Escribís el plano:

```

grid-template-areas:
  "cabecera cabecera"
  "menu contenido";

```

y eso **es** el dibujo. Dos filas (dos renglones de texto), dos columnas (dos palabras por renglón). La palabra **cabecera** aparece dos veces en el primer renglón: eso significa que la cabecera ocupa las dos columnas. Abajo, **menu** a la izquierda y **contenido** a la derecha.

Después, cada elemento dice a qué área pertenece con **grid-area: menu**, y Grid lo coloca. No hace falta calcular posiciones ni anchos: alcanza con que el dibujo y los nombres coincidan.

La unidad **fr**

fr quiere decir *fracción del espacio sobrante*. En **grid-template-columns: 120px 1fr**, Grid primero reserva 120px para la primera columna y después le da **todo lo que queda** a la segunda. No importa si la ventana mide 600px o 2000px: el menú siempre mide 120 y el contenido se adapta.

Comparalo con porcentajes: **20% 80%** funciona hasta que agregás un borde o un gap y la suma se pasa de 100. Con **fr** eso no puede pasar, porque se reparte lo que **realmente** sobra.

Paso a paso

BLOQUE 1 — El plano de la página

```
1 .panel {  
2   min-height: 100vh;  
3   display: grid;  
4   grid-template-columns: 120px 1fr;  
5   grid-template-areas:  
6     "cabecera cabecera"  
7     "menu contenido";  
8 }
```

Tres líneas, tres decisiones:

- `display: grid` convierte a `.panel` en contenedor de grilla. Sus tres hijos directos —cabecera, menú y contenido— pasan a ser celdas.
- `grid-template-columns: 120px 1fr` define **dos** columnas: una fija de 120px y otra que se queda con el resto.
- `grid-template-areas` dibuja el plano y, de paso, define implícitamente **dos filas**. No hizo falta escribir `grid-template-rows`: la cantidad de renglones del dibujo la determina.

El dibujo tiene que ser rectangular

Cada renglón de `grid-template-areas` debe tener **la misma cantidad de palabras**, y esa cantidad debe coincidir con la cantidad de columnas declaradas. Si escribís tres palabras en un renglón y dos en otro, o si declarás dos columnas y dibujás tres, Grid **ignora la propiedad entera** y no vas a ver ningún mensaje de error: simplemente no pasa nada.

Cuando el layout no aparece, contá las palabras. Es el error número uno de Grid.

Si una celda tiene que quedar vacía, se usa un punto: `"cabecera ."`.

Extra: por qué la cabecera se ve tan alta en una pantalla grande

Al abrir esta solución en una ventana alta vas a notar algo: la barra oscura de la cabecera queda **muchísimo más alta** de lo esperado. No es un error tuyo, y vale la pena entender por qué pasa.

`.panel` tiene `min-height: 100vh` (toda la altura de la ventana), pero nunca dijimos **cuánto mide cada fila**. Cuando no se dice, Grid las hace `auto` y reparte la altura sobrante **en partes iguales entre las dos**. Por eso la cabecera se infla.

La solución es una línea, y es la que usarías en un proyecto propio:

`grid-template-rows: auto 1fr;` → "la primera fila, lo que necesite; la segunda, todo el resto".

No está en la consigna y no hace falta para entregar el desafío. Pero si querés que el panel se vea como un panel de verdad, agregala debajo de `grid-template-columns`.

BLOQUE 2 — Cada elemento a su área

```

1  .cabecera {
2    padding: 18px;
3    background: var(--oscuro);
4    color: white;
5    grid-area: cabecera;
6  }
7
8  .menu {
9    padding: 18px;
10   line-height: 2;
11   background: #e8f1fb;
12   grid-area: menu;
13 }
14
15 .contenido {
16   padding: 20px;
17   grid-area: contenido;
18 }

```

Una línea por elemento. `grid-area: cabecera` se lee "este elemento va donde el plano dice *cabecera*". El nombre **no lleva comillas** acá (sí las lleva dentro de `grid-template-areas`) y tiene que estar escrito exactamente igual: `menu` sin tilde en los dos lados, o con tilde en los dos.

Fijate que la cabecera no necesita decir "ocupá dos columnas": eso ya está en el dibujo, donde la palabra aparece dos veces. El elemento solo dice a qué área pertenece.

BLOQUE 3 — La grilla de tarjetas

```

1  .tarjetas {
2    display: grid;
3    grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
4    gap: 12px;
5  }

```

Esta línea es, probablemente, la más potente de todo el curso de CSS. Vale la pena desarmarla de adentro hacia afuera:

Pieza	Qué hace
<code>minmax(120px, 1fr)</code>	Una columna que nunca baja de 120px y que, si hay lugar, crece hasta llenar su fracción.
<code>auto-fit</code>	"Poné tantas columnas como entren ". No decís cuántas: lo calcula el navegador con el ancho disponible.
<code>repeat(...)</code>	Repite esa definición de columna las veces que haga falta.
<code>gap: 12px</code>	Separación entre celdas, en filas y columnas, sin margen en los bordes exteriores.

El resultado: en una ventana ancha entran cuatro tarjetas por fila; al achicarla pasan a tres, a dos, a una. **Sin una sola media query**. Es lo que se llama diseño intrínsecamente responsivo: la regla no describe pantallas, describe una

intención ("tarjetas de al menos 120px, las que entren"), y el navegador resuelve.

BLOQUE 4 — La tarjeta destacada

```
1 .destacada {
2   border-color: var(--azul) !important;
3   grid-column: span 2;
4 }
```

`grid-column: span 2` se lee "ocupá dos columnas a partir de donde te toque". No dice *cuáles* dos columnas: dice *cuántas*. Por eso sigue funcionando aunque la cantidad de columnas cambie con el ancho de la ventana.

El caso límite lo resuelve Grid solo

¿Qué pasa si la ventana se angosta tanto que `auto-fit` deja una sola columna, y la tarjeta destacada pide dos? Grid no rompe nada: le da el ancho completo disponible. Ese comportamiento tolerante es una de las razones por las que Grid se volvió la herramienta estándar para layouts.

Sobre el `!important` del color de borde: viene del archivo original y está ahí para ganarle a la regla `.tarjetas article`, que es más específica (`.tarjetas article` vale más que `.destacada`). En un proyecto propio la solución elegante sería escribir `.tarjetas .destacada` y evitar el `!important`, que conviene reservar para casos excepcionales.

Solución completa

css/estilos.css — solución completa

CSS

```
1  /* =====
2    Desafío Clase 10 — Panel de proyectos
3    css/estilos.css
4    -----
5    DOS GRILLAS ANIDADAS
6    .panel es una grilla que ubica cabecera, menú y contenido.
7    .tarjetas es OTRA grilla, adentro del contenido, que acomoda las
8        tarjetas.
9    Son independientes: las columnas de una no afectan a las de la otra.
10   Todo se resuelve con Grid: sin Flexbox, sin position, sin media
11   queries.
12   ===== */
13
14  :root {
15    --oscuro: #1d1d1f;
16    --fondo: #f5f5f7;
17    --azul: #0057b8;
18  }
19
20  * {
21    box-sizing: border-box;
22  }
23
24  body {
```

```

25     margin: 0;
26     font-family: Arial, sans-serif;
27     background: var(--fondo);
28 }
29
30 /* ---- BLOQUE 1: la grilla de la página ----- */
31 /* Menú de 120px fijos y contenido con todo el resto.
32    El dibujo tiene dos filas: la cabecera cruza las dos columnas,
33    y abajo el menú y el contenido comparten la fila. */
34 .panel {
35     min-height: 100vh;
36     display: grid;
37     grid-template-columns: 120px 1fr;
38     grid-template-areas:
39         "cabecera cabecera"
40         "menu contenido";
41 }
42
43 /* ---- BLOQUE 2: cada elemento a su área ----- */
44 .cabecera {
45     padding: 18px;
46     background: var(--oscuro);
47     color: white;
48     grid-area: cabecera;
49 }
50
51 .menu {
52     padding: 18px;
53     line-height: 2;
54     background: #e8f1fb;
55     grid-area: menu;
56 }
57
58 .contenido {
59     padding: 20px;
60     grid-area: contenido;
61 }
62
63 /* ---- BLOQUE 3: la grilla interna de tarjetas ----- */
64 /* Columnas automáticas: se adapta al ancho que le deje el
65     .contenido, que a su vez depende del ancho de la ventana.
66     Las dos grillas se ajustan en cadena sin ninguna media query. */
67 .tarjetas {
68     display: grid;
69     grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
70     gap: 12px;
71 }
72
73 .tarjetas article {
74     padding: 18px;
75     border: 1px solid #d2d2d7;
76     border-radius: 8px;
77     background: white;
78 }
79
80 .tarjetas p {
81     margin-bottom: 0;
82     color: #6e6e73;
83 }
84
85 /* ---- BLOQUE 4: la tarjeta destacada ----- */

```

```

86  /* Resumen ocupa dos celdas de la grilla de tarjetas.
87     Si la pantalla se angosta tanto que solo entra una columna,
88     Grid lo resuelve solo y la tarjeta ocupa el ancho completo. */
89  .destacada {
90    border-color: var(--azul) !important;
91    grid-column: span 2;
92  }

```

Cómo probarlo

1. La cabecera oscura tiene que cruzar **todo el ancho**, por encima del menú y del contenido.
2. El menú celeste, angosto y a la izquierda; el contenido, a la derecha.
3. El menú tiene que llegar hasta abajo del todo (por eso `min-height: 100vh` en `.panel`).
4. Achicá y agrandá la ventana lentamente: las tarjetas tienen que cambiar de cantidad por fila. Contá cuántas hay en cada tamaño.
5. "Resumen" tiene que ocupar siempre el doble de ancho que las demás, salvo cuando queda una sola columna.
6. En las DevTools, buscá el ícono `grid` al lado de `display: grid` y actívalo: el navegador dibuja las líneas de la grilla, numera las columnas y muestra los nombres de las áreas encima de la página. Es la forma más rápida de ver si tu dibujo coincide con lo que esperabas.

Errores frecuentes

Qué pasa	Por qué	Cómo se arregla
El layout no cambia nada, todo apilado	El dibujo de <code>grid-template-areas</code> no es rectangular, o no coincide con la cantidad de columnas	Contá las palabras de cada renglón: todas iguales, y tantas como columnas declaraste.
Un elemento aparece en el lugar equivocado	El nombre del <code>grid-area</code> no coincide exactamente con el del dibujo	Revisá mayúsculas, tildes y guiones. <code>menú</code> y <code>menu</code> son distintos.
Las tarjetas quedan una debajo de otra	Falta <code>display: grid</code> en <code>.tarjetas</code>	<code>grid-template-columns</code> sin <code>display: grid</code> no hace nada.
Las tarjetas no cambian de cantidad al achicar	Se usó <code>repeat(4, 1fr)</code> en vez de <code>repeat(auto-fit, minmax(...))</code>	Con un número fijo, siempre hay cuatro columnas y se aprietan.
Aparece scroll horizontal	Falta <code>box-sizing: border-box</code> , o se usaron porcentajes que suman más de 100 con el gap	Con <code>fr</code> y <code>gap</code> esto no debería pasar.
Se intentó centrar con Flexbox y quedó raro	La consigna pide resolverlo solo con Grid	Todo lo que necesitás está en Grid: áreas, <code>fr</code> y <code>span</code> .

Grid o Flexbox: cómo elegir

La regla práctica: **Grid para el plano de la página, Flexbox para acomodar cosas adentro de un componente**. Grid es bidimensional y se piensa desde el contenedor hacia afuera ("así se ve la página"); Flexbox es unidimensional y se piensa desde el contenido ("estas cosas van en fila"). No compiten: en un proyecto real se usan los dos, a menudo anidados uno adentro del otro.

CAPÍTULO 6 · CLASE 11

Desafío: Adaptar un panel de seguimiento

La misma página tiene que funcionar en un celular, en una tablet y en una pantalla de escritorio. Tres etapas, dos media queries y una regla que no se negocia: empezar por el celular.

Ficha del desafío

Clase	Clase 11 — Responsive desde cero: adaptar una página al ancho
Desafío	Adaptar un panel de seguimiento
Qué se practica	<code>@media (min-width)</code> , enfoque mobile first, la etiqueta <code>viewport</code> , y cómo cambia un layout de Grid entre etapas
Archivos del proyecto	<code>index.html</code> + <code>css/estilos.css</code>
Qué se toca	Solo los cuatro bloques marcados, dentro de las dos media queries.

Estructura del proyecto

```
desafio-clase-11/
├─ index.html
├─ css/
│   └─ estilos.css
```

El archivo `index.html`

Acá el `<head>` tiene una etiqueta que en los desafíos anteriores era una buena costumbre y en este es imprescindible.

index.html		HTML
1	<code><!DOCTYPE html></code>	
2	<code><html lang="es"></code>	
3	<code><head></code>	
4	<code><meta charset="UTF-8"></code>	
5		
6	<code><!-- SIN esta línea, las media queries NO funcionan en un</code>	

```

7      celular real: el teléfono finge ser una pantalla ancha
8      y achica todo. Es obligatoria en cualquier página
9      responsiva. -->
10     <meta name="viewport" content="width=device-width, initial-scale=1.0">
11
12     <title>Panel responsivo</title>
13     <link rel="stylesheet" href="css/estilos.css">
14 </head>
15 <body>
16     <div class="panel">
17
18         <header class="cabecera">Panel de seguimiento</header>
19
20         <aside class="menu">
21             <strong>UDI</strong>
22             <p>Resumen</p>
23             <p>Proyectos</p>
24             <p>Equipo</p>
25         </aside>
26
27         <main class="contenido">
28             <h1>Resumen</h1>
29
30             <section class="metricas">
31                 <article><strong>12</strong><span>Proyectos</span></article>
32                 <article><strong>8</strong><span>Pendientes</span></article>
33                 <article><strong>6</strong><span>Personas</span></article>
34                 <article><strong>3</strong><span>Entregas</span></article>
35             </section>
36
37             <section class="trabajo">
38                 <article>
39                     <h2>Actividad</h2>
40                     <p>Se completaron cinco tareas esta semana.</p>
41                 </article>
42                 <article>
43                     <h2>Próxima entrega</h2>
44                     <p>Viernes, 18:00.</p>
45                 </article>
46             </section>
47         </main>
48
49     </div>
50 </body>
51 </html>

```

Sin viewport, nada de esto funciona en un teléfono

Un celular tiene una pantalla de, digamos, 390px de ancho real. Pero por compatibilidad con sitios viejos, los navegadores móviles **fingen** tener unos 980px y después achican todo el resultado como si fuera una foto. Con ese engaño activo, una media query de `min-width: 900px` se cumple en un celular, y tu página se ve en modo escritorio, diminuta.

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` le dice "usá tu ancho real y no achiques nada". Es la línea que hace posible el diseño responsivo, y se escribe siempre igual.

La consigna

CONSIGNA TAL COMO APARECE EN LA PLATAFORMA

OBJETIVO

Completar un panel que se organiza en tres etapas: celular, pantalla mediana y escritorio.

RESULTADO ESPERADO

- Base: menú oculto, métricas y paneles en una columna.
- Desde 600px: métricas en dos columnas.
- Desde 900px: menú lateral, cuatro métricas en una fila y trabajo en dos columnas.

BLOQUE 1 – 600PX

En la primera media query, hacé que `.metricas` tenga dos columnas iguales.

BLOQUE 2 – ESTRUCTURA DE ESCRITORIO

En `.panel` activá Grid, creá columnas de 180px y 1fr, y dibujá:

```
"cabecera cabecera"
"menu contenido"
```

BLOQUE 3 – ASIGNAR ÁREAS

Asigná cabecera, menu y contenido con `grid-area`. Mostrá `.menu` con `display: block`.

BLOQUE 4 – CONTENIDO

En escritorio, `.metricas` debe tener cuatro columnas. `.trabajo` debe tener columnas 2fr y 1fr.

CÓMO COMPROBARLO

- En la previsualización angosta todo se lee en una columna y el menú no ocupa espacio.
- Desde 600px las métricas forman dos columnas.
- Desde 900px el dibujo de áreas coincide con la pantalla.
- No hay elementos superpuestos ni scroll horizontal.
- No usaste JavaScript ni media queries con `max-width`.

Punto de partida del CSS

El archivo ya trae resuelta la etapa 1 (el celular) y tiene las dos media queries armadas, vacías por dentro.

css/estilos.css – punto de partida (recortado)

CSS

```
1  /* — BASE: CELULAR (sin media query) ————— */
2  .panel {
3    min-height: 100vh;
4  }
5  .menu {
6    display: none;          /* en celular el menú no se muestra */
7    padding: 20px;
8    background: #e8f1fb;
9  }
10 .metricas, .trabajo {
11   display: grid;
```

```

12   grid-template-columns: 1fr; /* una sola columna */
13   gap: 12px;
14 }
15
16 /* — ETAPA 2 — */
17 @media (min-width: 600px) {
18   .metricas {
19     /* BLOQUE 1 */
20   }
21 }
22
23 /* — ETAPA 3 — */
24 @media (min-width: 900px) {
25   .panel {
26     /* BLOQUE 2 */
27   }
28   .cabecera {
29     /* BLOQUE 3 */
30   }
31   .menu {
32     /* BLOQUE 3 */
33   }
34   .contenido {
35     /* BLOQUE 3 */
36   }
37   .metricas {
38     /* BLOQUE 4 */
39   }
40   .trabajo {
41     /* BLOQUE 4 */
42   }
43 }

```

Cómo lo encara un desarrollador

Qué es una media query

Una media query es una condición: "si se cumple esto, aplicá estas reglas". `@media (min-width: 600px)` se lee "de 600px de ancho **para arriba**". Las reglas de adentro solo existen cuando la condición se cumple; el resto del tiempo es como si no estuvieran escritas.

Los anchos donde se hace un cambio se llaman **breakpoints**. No hay una lista oficial: se eligen mirando **dónde el diseño empieza a verse mal**, no según modelos de teléfono. Acá son 600px (donde ya entran dos métricas cómodas) y 900px (donde hay lugar para un menú lateral sin ahogar el contenido).

Mobile first: por qué solo `min-width`

La consigna prohíbe `max-width`, y no es una regla arbitraria. Hay dos formas de escribir CSS responsivo:

Mobile first — min-width	Desktop first — max-width
El CSS base describe el celular . Cada media query agrega cosas a medida que hay más espacio.	El CSS base describe el escritorio . Cada media query deshace cosas a medida que hay menos espacio.
Las reglas se acumulan: cada etapa parte de la anterior.	Las reglas se cancelan entre sí: aparecen <code>display: block</code> para anular un <code>grid</code> , <code>1fr</code> para anular un <code>repeat(4, 1fr)</code> ...
El celular —que suele ser la mayoría del tráfico y el dispositivo más lento— carga el CSS más simple.	El celular tiene que procesar primero todo el layout de escritorio y después las reglas que lo deshacen.

Por eso el archivo empieza describiendo la versión más simple posible —una columna, menú oculto— y va sumando. En la etapa 3 no hay una sola regla que "apague" algo de la etapa 2.

Paso a paso

Etapla 1 — La base (ya viene resuelta)

Vale la pena mirarla, porque es el punto de partida de todo lo demás. Tres decisiones:

- `.menu { display: none; }` — en un celular no hay lugar para un menú lateral. `display: none` lo saca de la página y del flujo: no ocupa lugar ni deja un hueco. (Distinto de `visibility: hidden`, que lo esconde pero le sigue reservando el espacio.)
- `.metricas, .trabajo { grid-template-columns: 1fr; }` — una sola columna. Todo apilado, que es lo correcto en una pantalla angosta.
- La cabecera y el contenido no necesitan nada especial: por defecto los elementos de bloque ya ocupan todo el ancho y se apilan.

BLOQUE 1 — Etapa 2, desde 600px

```

1  @media (min-width: 600px) {
2    .metricas {
3      grid-template-columns: 1fr 1fr;
4    }
5  }
```

Una sola declaración. A 600px ya hay lugar para dos métricas por fila sin que los números y las etiquetas queden apretados. Fijate que **no hace falta repetir** `display: grid` ni `gap`: eso ya lo puso la regla base y sigue vigente. La media query solo escribe **lo que cambia**.

Ese es el corazón del enfoque mobile first: cada etapa es un ajuste sobre la anterior, no una versión nueva completa.

BLOQUE 2 — Etapa 3, la estructura de escritorio

```

1  @media (min-width: 900px) {
2    .panel {
3      display: grid;
4      grid-template-columns: 180px 1fr;
5      grid-template-areas:
6        "cabecera cabecera"
7        "menu contenido";
8    }
9  }

```

Recién a los 900px el panel se convierte en una grilla. Antes de ese ancho, `.panel` era un simple contenedor y sus hijos se apilaban solos. Es exactamente el mismo dibujo de áreas del desafío de la clase 10, con el menú un poco más ancho (180px en vez de 120px).

Extra: la misma cabecera alta que en la clase 10

Igual que en el desafío anterior, en una pantalla alta la cabecera se infla: `.panel` tiene `min-height: 100vh` y Grid reparte la altura sobrante entre sus dos filas, porque no le dijimos cuánto mide cada una.

Se arregla con una línea dentro de la media query de 900px: `grid-template-rows: auto 1fr;` — la cabecera lo que necesite, el resto para el cuerpo. No está en la consigna; es un extra para dejarlo prolijo.

Una idea que vale la pena masticar

Grid y las media queries se combinan de una forma muy potente: **un mismo HTML puede tener planos completamente distintos según el ancho**, sin mover una etiqueta de lugar. En celular, `.panel` no es una grilla y todo se apila. En escritorio, es una grilla de dos columnas con áreas nombradas. El HTML no se enteró de nada.

BLOQUE 3 — Asignar las áreas (y mostrar el menú)

```

1  .cabecera {
2    grid-area: cabecera;
3  }
4
5  .menu {
6    grid-area: menu;
7    display: block;    /* las dos cosas son necesarias */
8  }
9
10 .contenido {
11   grid-area: contenido;
12 }

```

La cabecera y el contenido solo necesitan su `grid-area`. El menú necesita **dos declaraciones**, y esta es la parte donde más gente se traba:

- `grid-area: menu` le dice **dónde** ubicarse.
- `display: block` le dice que **exista**, anulando el `display: none` de la etapa base.

Falta cualquiera de las dos y no se ve. Con solo `grid-area`, el elemento sigue en `display: none` y no se dibuja. Con solo `display: block`, aparece pero Grid lo mete en la primera celda libre y desordena todo.

BLOQUE 4 — El contenido en escritorio

```

1  .metricas {
2    grid-template-columns: repeat(4, 1fr);
3  }
4
5  .trabajo {
6    grid-template-columns: 2fr 1fr;
7  }

```

Las cuatro métricas en una fila: `repeat(4, 1fr)` es la forma corta de escribir `1fr 1fr 1fr 1fr`. Acá el número es fijo a propósito —queremos exactamente cuatro, no "las que entren"— porque son cuatro indicadores que se leen mejor juntos.

Y en `.trabajo`, la decisión más interesante del desafío: `2fr 1fr` en vez de `1fr 1fr`. Las dos tarjetas no tienen la misma cantidad de texto: "Actividad" tiene una oración entera y "Próxima entrega" tiene tres palabras. Con columnas iguales, la segunda queda medio vacía y la primera apretada. Con 2fr y 1fr, la primera se lleva dos tercios y el conjunto queda visualmente equilibrado.

El detalle que separa "funciona" de "está bien hecho"

La consigna pedía `2fr` y `1fr` sin explicar por qué. Ese "por qué" es una decisión de diseño: **el ancho de una columna debería responder a cuánto contenido lleva**, no a una simetría abstracta. Es el tipo de criterio que no se aprende de una propiedad, sino de mirar el resultado y preguntarse si se ve equilibrado.

Solución completa

css/estilos.css — solución completa

CSS

```

1  /* =====
2   Desafío Clase 11 – Panel de seguimiento responsivo
3   css/estilos.css
4   -----
5   TRES ETAPAS, MOBILE FIRST
6   Base (sin media query) → celular: todo en una columna, menú oculto.
7   @media 600px           → tablet: métricas de a dos.
8   @media 900px           → escritorio: menú lateral, cuatro métricas
9                           en fila y el trabajo en dos columnas.
10
11   Se usa solo min-width: cada etapa AGREGA sobre la anterior, nunca
12   deshace. Con max-width habría que escribir reglas que cancelan
13   otras reglas, que es de donde salen la mayoría de los bugs.
14   ===== */
15
16  :root {
17    --oscuro: #1d1d1f;
18    --fondo: #f5f5f7;
19    --panel: #ffffff;

```

```
20  --azul: #0057b8;
21  }
22
23  * {
24    box-sizing: border-box;
25  }
26
27  body {
28    margin: 0;
29    font-family: Arial, sans-serif;
30    background: var(--fondo);
31    color: var(--oscuro);
32  }
33
34  /* — ETAPA 1 — BASE: CELULAR ————— */
35
36  .panel {
37    min-height: 100vh;
38  }
39
40  .cabecera {
41    padding: 18px;
42    background: var(--oscuro);
43    color: white;
44  }
45
46  /* En celular el menú no se muestra y, al estar en display: none,
47     tampoco ocupa lugar. */
48  .menu {
49    display: none;
50    padding: 20px;
51    background: #e8f1fb;
52  }
53
54  .contenido {
55    padding: 20px;
56  }
57
58  /* Una sola columna: todo apilado, que es lo correcto en un
59     teléfono. */
60  .metricas, .trabajo {
61    display: grid;
62    grid-template-columns: 1fr;
63    gap: 12px;
64  }
65
66  .metricas article, .trabajo article {
67    padding: 18px;
68    border-radius: 8px;
69    background: var(--panel);
70    border: 1px solid #d2d2d7;
71  }
72
73  .metricas {
74    margin-bottom: 12px;
75  }
76
77  .metricas strong, .metricas span {
78    display: block;
79  }
80
```

```

81 .metricas strong {
82   color: var(--azul);
83   font-size: 26px;
84 }
85
86 /* — ETAPA 2 — DESDE 600px ————— */
87 @media (min-width: 600px) {
88
89   /* BLOQUE 1: las cuatro métricas pasan a dos por fila.
90    Es el primer nivel donde entra más de una sin apretarse. */
91   .metricas {
92     grid-template-columns: 1fr 1fr;
93   }
94 }
95
96 /* — ETAPA 3 — DESDE 900px ————— */
97 @media (min-width: 900px) {
98
99   /* BLOQUE 2: recién acá el panel se vuelve grilla.
100    El dibujo pone la cabecera cruzando las dos columnas, y
101    debajo el menú a la izquierda y el contenido a la derecha. */
102   .panel {
103     display: grid;
104     grid-template-columns: 180px 1fr;
105     grid-template-areas:
106       "cabecera cabecera"
107       "menu contenido";
108   }
109
110   /* BLOQUE 3 */
111   .cabecera {
112     grid-area: cabecera;
113   }
114
115   /* Se muestra Y se ubica en su área. Las dos cosas son
116    necesarias: sin display: block seguiría oculto; sin
117    grid-area no sabría en qué celda ponerse. */
118   .menu {
119     grid-area: menu;
120     display: block;
121   }
122
123   .contenido {
124     grid-area: contenido;
125   }
126
127   /* BLOQUE 4: con el ancho de escritorio las cuatro métricas
128    entran cómodas en una sola fila. */
129   .metricas {
130     grid-template-columns: repeat(4, 1fr);
131   }
132
133   /* 2fr y 1fr, no 1fr 1fr. "Actividad" tiene más texto que
134    "Próxima entrega", así que se queda con dos tercios del
135    ancho y las dos tarjetas quedan visualmente equilibradas. */
136   .trabajo {
137     grid-template-columns: 2fr 1fr;
138   }
139 }

```

Cómo probarlo

Este desafío se prueba **cambiando el ancho**, así que las DevTools son imprescindibles.

1. Abrí la página y apretá **F12**. Después, el ícono de celular/tablet arriba a la izquierda del panel (o **Ctrl+Shift+M**): eso activa el **modo dispositivo**, que te deja fijar el ancho exacto.
2. Poné el ancho en **375px** (un celular típico). Tiene que verse todo en una columna y el menú no tiene que aparecer **ni dejar un hueco**.
3. Subí a **700px**. Las métricas tienen que pasar a dos por fila. El menú sigue oculto.
4. Subí a **1200px**. Ahora sí: menú lateral a la izquierda, cabecera cruzando arriba, cuatro métricas en una fila y las dos tarjetas de trabajo con anchos distintos.
5. Pasá lentamente por **599 → 600** y por **899 → 900**. En cada uno tiene que verse el salto. Si no pasa nada en 600, revisá esa media query.
6. En cualquier ancho, la página **nunca** tiene que tener scroll horizontal.

Errores frecuentes

Qué pasa	Por qué	Cómo se arregla
En el celular real se ve la versión de escritorio, chiquita	Falta la etiqueta <code>viewport</code> en el <code><head></code>	Es el error más importante del desafío. En la computadora no se nota; en un teléfono, sí.
El menú no aparece nunca	Se puso <code>grid-area</code> pero falta <code>display: block</code>	Hacen falta las dos declaraciones en la media query de 900px.
El menú aparece pero desordena todo	Se puso <code>display: block</code> pero falta <code>grid-area</code>	Sin área asignada, Grid lo ubica en la primera celda libre.
Nada cambia al achicar la ventana	La media query está escrita afuera de todo, o le falta una llave de cierre	Contá las llaves: una abre el <code>@media</code> y otra cierra cada regla de adentro.
Los cambios se pisan entre sí	Las media queries están en el orden equivocado (900 antes que 600)	Con <code>min-width</code> , van de menor a mayor. La de arriba es la más chica.
Aparece scroll horizontal en celular	Algún elemento tiene un ancho fijo mayor que la pantalla	Buscalo en las DevTools: en modo dispositivo, el elemento culpable suele sobresalir visiblemente.

Para cerrar la serie

Este es el último desafío de la parte de CSS, y no es casualidad que combine todo: el Box Model de la clase 7 (para que nada se desborde), las variables de la clase 8 (los colores del panel), Grid de la clase 10 (el plano de la página) y las media queries de esta clase.

Si podés armar este proyecto desde cero en tu computadora, con sus dos archivos, y explicar por qué cada media query dice lo que dice, ya sabés maquetar una página real. Lo que sigue —JavaScript— es hacer que esa página *haga* cosas.